Pairing-Based Verifiable Shuffles with Logarithmic-Size Proofs

Yuxi Xue, Xingye Lu, and Man Ho Au

The Hong Kong Polytechnic University, Hong Kong
yuxi-ivy.xue@connect.polyu.hk
{mhaau,xing-ye.lu}@polyu.edu.hk

Abstract. A verifiable shuffle proves that a list of output ciphertexts is a rerandomized permutation of a list of input ciphertexts, without revealing either the permutation or the rerandomization factors. Verifiable shuffles are a core primitive in mix-nets and are deployed in national electronic voting systems and blockchain-based anonymization protocols. Existing deployed verifiable shuffles typically have proof size $O(N)$ or $O(\sqrt{N})$ in the number of ciphertexts N, making shuffle proofs a primary bandwidth cost. The only prior construction with $O(\log N)$ proof size (Hoffmann et al., CCS 2019) requires roughly $30N$ prover and $10N$ verifier group exponentiations, with a proof consisting of $6 \log N + 8$ group elements and 4 field elements.

In this paper, we present a new verifiable shuffle for ElGamal ciphertexts whose proof consists of $2 \log N + 8$ group elements and 8 field elements, reducing the prover and verifier costs of Hoffmann et al. to $15N$ and $6N$ group exponentiations, respectively. Our protocol is public-coin, non-interactive via the Fiat-Shamir transform, and relies on an updatable structured reference string generated once in a powers-of-tau ceremony and reusable across applications.

We implement the protocol and, to the best of our knowledge, provide the first benchmarks for a verifiable shuffle with logarithmic proof size. At $N = 2^{20}$ (about one million ciphertexts), the proof is only 2.5 KB, compared with hundreds of kilobytes for the best $O(\sqrt{N})$-size scheme and hundreds of megabytes for representative $O(N)$-size schemes.

Keywords: Zero-knowledge proofs · Verifiable shuffles · E-voting.

1 Introduction

Mix-nets, introduced by Chaum [10], are a foundational tool for anonymous communication. In a mix-net, a sequence of servers each receives a batch of encrypted messages, permutes the order, applies fresh randomness to re-encrypt them, and forwards the result to the next server. As long as at least one server is honest, the correspondence between inputs and outputs remains hidden. This simple paradigm has found applications across a wide range of privacy-sensitive settings. In e-voting [17,4,3], mix-nets are used to break the link between voters and their ballots: encrypted votes are shuffled through a sequence of mix-servers,

after which they can be decrypted and published without revealing which voter cast which ballot. Mix-net-based systems have been deployed in national elections in countries including Norway and Switzerland [38,27], routinely handling shuffles of several hundred thousand encrypted ballots. More recently, mix-net shuffles have been adopted in blockchain-based coin mixing protocols such as MixEth [35] for anonymization of cryptocurrency transactions on Ethereum.

A central challenge in these applications is verifiability: the party responsible for the shuffle may modify, drop, or substitute ciphertexts, thereby compromising correctness. In the standard mix-net setting, this is addressed by requiring each server to produce a zero-knowledge proof that its outputs are a rerandomized permutation of its inputs. A protocol that provides such a proof is called a verifiable shuffle, and constructing efficient verifiable shuffles has been one of the central topics in cryptographic protocol design.

The efficiency of a verifiable shuffle is typically measured by proof size, prover time, and verifier time. Among these, proof size plays a distinguished role in practice. In e-voting, each mix-server publishes its proof on a public bulletin board, and any auditor must obtain all such proofs. With k servers, the total proof volume scales linearly in k. In blockchain-based protocols such as MixEth [35], the situation is more extreme: shuffle proofs are posted on-chain, where on-chain storage is particularly expensive. A reduction in proof size therefore, translates directly into lower gas costs and higher throughput.

A natural way to build verifiable shuffle is to rely on generic zkSNARKs, such as PLONK [19] and Groth16 [23], to write the shuffle relation as an arithmetic circuit and prove in zero knowledge that the circuit is satisfied. For shuffling, however, this approach typically incurs substantial overhead. First, routing N ciphertexts through a secret permutation requires a permutation network of size $\Omega(N \log N)$. Second, the shuffle relation is defined over an elliptic curve group. When the zkSNARK's native field does not match the curve's field, these operations must be implemented via non-native arithmetic inside the circuit, leading to substantial constraint overhead. Dedicated shuffle arguments avoid these costs by working natively with group elements and exploiting the algebraic structure of the shuffle relation.

Early verifiable shuffle protocols [18,32,22,36] incur $O(N)$ communication, where N denotes the number of ciphertexts. Groth's first sublinear construction [24] reduced the proof size to $O(\sqrt{N})$, and the subsequent refinement of Bayer and Groth [6] became one of the most widely deployed approaches in this regime. Hoffmann et al. [28] later combined the Bayer–Groth framework [6] with inner product arguments [7,9] to obtain the first verifiable shuffle with $O(\log N)$ proof size, in the plain discrete-log setting and with a transparent setup. To our knowledge, this is the only prior dedicated verifiable-shuffle construction achieving $O(\log N)$ proof size. Despite this asymptotic improvement, two limitations remain. First, the prover requires approximately $30N$ group exponentiations and the verifier $10N$, leaving room for improvement in prover and verifier efficiency. Second, no implementation of this scheme has been reported, and the deployed

Table 1: Comparison of verifiable shuffle protocols for N ciphertexts ($N = mn$ where applicable). "exp" denotes a group exponentiation in $\mathbb{G}_1$, and "p" denotes a pairing computation. A uniform setup requires only publicly verifiable randomness. A structured setup requires a structured reference string (SRS) generated together with trapdoor information that must be discarded after setup. An updatable setup is a structured setup in which any party may extend the ceremony by contributing fresh randomness, and security holds as long as at least one participant is honest.

Scheme	**Proof Size**	**Prover**	**Verifier**	**Setup**
[18]	$5N\,\mathbb{G}_1 + N\,\mathbb{F}$	$8N$ exp	$10N$ exp	Uniform
[17]	$5N\,\mathbb{G}_1 + N\,\mathbb{F}$	$9N$ exp	$10N$ exp	Uniform
[22]	$3N\,\mathbb{F}$	$6N$ exp	$6N$ exp	Uniform
[16]	$N\,\mathbb{G}_1 + 2N\,\mathbb{F}$	$7N$ exp	$8N$ exp	Uniform
[36]	$3N\,\mathbb{G}_1 + 4N\,\mathbb{F}$	$9N$ exp	$11N$ exp	Uniform
[30]	$\sim N\,\mathbb{G}_1 + 2N\,\mathbb{F}$	$6N$ exp	$6N$ exp	Uniform
[11]	$7N\,\mathbb{G}_1$	$16N$ exp	$18N$ p	Structured
[12]	$\sim 7N\,\mathbb{G}_1$	$\sim 7N$ exp	$\sim 3N$ exp + $3N$ p	Structured
[24]	$3m^2\,\mathbb{G}_1 + 3n\,\mathbb{F}$	$3mN$ exp	$4N$ exp	Uniform
[6]	$11m\,\mathbb{G}_1 + 5n\,\mathbb{F}$	$\sim 6N \log m$ exp	$4N$ exp	Uniform
[28]	$(6\log N{+}8)\,\mathbb{G}_1 + 4\,\mathbb{F}$	$\sim 30N$ exp	$\sim 10N$ exp	Uniform
This work	$(2\log N{+}8)\,\mathbb{G}_1 + 8\,\mathbb{F}$	$15N$ exp	$6N$ exp + 3 p	Structured (updatable)

systems [38,27] therefore continue to rely on schemes with $O(N)$ or $O(\sqrt{N})$ proof size. A more detailed survey of prior work appears in Section 2 and Table 1.

This work addresses both limitations. Compared with Hoffmann et al. [28], we achieve a smaller proof and reduce the prover and verifier group exponentiation counts by roughly half, at the cost of a universal structured reference string (SRS) in place of a transparent setup. The SRS is generated once via a powers-of-tau ceremony [8], is updatable, and can be reused for any shuffle of up to N ciphertexts. In addition, we provide the first implementation of a logarithmic-size verifiable shuffle, together with concrete benchmarks. Specifically, we summarize our contribution as following:

1. We construct a verifiable shuffle protocol for ElGamal ciphertexts with proof size $(2\log N + 8)\,\mathbb{G}_1 + 8\,\mathbb{F}$, $15N$ group exponentiations for the prover, and $6N$ group exponentiations plus 3 pairings for the verifier. The protocol becomes non-interactive via the Fiat-Shamir transform. Compared with [28], it reduces the $\log N$ coefficient in proof size by a factor of three, the prover cost from $\sim 30N$ to $15N$, and the verifier's from $\sim 10N$ to $6N$ exponentiations.
2. We implement our protocol in Rust on BLS12-381 and, to the best of our knowledge, provide the first concrete benchmarks for an $O(\log N)$-size verifiable shuffle. At $N = 2^{20}$, our proof size is 2.5 KB. For comparison at the same N, the $O(\log N)$-size scheme [28] gives a proof size of 4.13 KB (1.65×

larger), while representative $O(\sqrt{N})$- and $O(N)$-size schemes [6,36] give 512 KB and 224 MB, respectively.

1.1 Technical Overview

A common strategy in verifiable shuffle design [32,22,6,28] is to decompose the proof into two relations: first, the prover knows a valid permutation of the input ciphertexts, and second, applying this permutation together with suitable rerandomization factors transforms the inputs into the output ciphertexts. We adopt the same high-level approach for ElGamal ciphertexts, the standard ciphertext type in mix-net-based e-voting and coin mixing.

At a more concrete level, our construction and that of Hoffmann et al. [28] differ in how these two relations are proven. For the permutation relation, Hoffmann et al. reduce it to a product relation and prove it using inner-product arguments (IPAs). We instead adopt the permutation argument of [19], based on the KZG polynomial commitment scheme, which yields a constant-size proof via a single batched pairing equation. For the consistency argument, Hoffmann et al. formulate the relation as a linear map and again prove it with an IPA. We instead express consistency directly as three ElGamal relations, aggregate them into a single inner-product instance via a random challenge, and prove the result with a single invocation of an IPA. As a result, our proof is more compact overall since the permutation component is constant-size, and consistency requires only one logarithmic-size IPA proof.

Concretely, let N be the number of ciphertexts to be shuffled. The public statement consists of input and output ciphertext vectors $\mathbf{C} := (\mathsf{C}_1, \ldots, \mathsf{C}_N)$ and $\tilde{\mathbf{C}} := (\tilde{\mathsf{C}}_1, \ldots, \tilde{\mathsf{C}}_N)$, where each $\mathsf{C}_i, \tilde{\mathsf{C}}_i \in \mathbb{G}_1^2$ is an ElGamal ciphertext. The prover's witness consists of a permutation $\pi \in \Sigma_N$ and rerandomization factors $\boldsymbol{\rho} \in \mathbb{F}^N$ such that

$$\tilde{\mathsf{C}}_i = \mathsf{C}_{\pi(i)} \cdot \mathsf{Enc}_{pk}(1; \rho_i) \qquad \text{for all } i \in [N]. \tag{1}$$

The goal is to convince the verifier that $\tilde{\mathbf{C}}$ is a rerandomized permutation of $\mathbf{C}$ while revealing nothing about π or $\boldsymbol{\rho}$. Following [32,22,6], we encode shuffle correctness by sampling a random challenge $x \in \mathbb{F}$ and using the powers $x^1, \ldots, x^N$ as position-dependent weights. If the shuffle relation (1) holds, then

$$\prod_{i=1}^{N} \mathsf{C}_i^{x^i} = \prod_{i=1}^{N} \tilde{\mathsf{C}}_i^{x^{\pi(i)}} \cdot \mathsf{Enc}_{pk}(1; -\rho_i)^{x^{\pi(i)}}. \tag{2}$$

Under a correct shuffle, these weights are merely reordered by the secret permutation. Writing $a_i := x^{\pi(i)}$, the prover is thus reduced to convincing the verifier that the vector $\boldsymbol{a} = (a_1, \ldots, a_N)$ satisfies:

(i) <u>Permutation:</u> $\boldsymbol{a}$ is a permutation of $(x, \ldots, x^N)$, and
(ii) <u>Consistency:</u> $\boldsymbol{a}$ and $\boldsymbol{\rho}$ satisfy $\prod_{i=1}^{N} \mathsf{C}_i^{x^i} = \prod_{i=1}^{N} \tilde{\mathsf{C}}_i^{a_i} \cdot \mathsf{Enc}_{pk}(1; -\rho_i)^{a_i}$.

We enforce these two properties with separate arguments that share the same committed vector $\boldsymbol{a}$.

Specifically, let $H = \{\omega, \omega^2, \ldots, \omega^N = 1\}$ be a multiplicative subgroup of $\mathbb{F}$, where ω is a primitive N-th root of unity, and let $\mathcal{L}_i(X)$ denote the i-th Lagrange basis polynomial over H, i.e., the unique degree-$(N-1)$ polynomial satisfying $\mathcal{L}_i(\omega^j) = 1$ if $i = j$ and 0 otherwise. The prover encodes $\boldsymbol{a}$ as the polynomial $f^*(X) = \sum_{i=1}^{N} a_i \mathcal{L}_i(X)$ and sends a hiding KZG commitment C_f to f^*.

Core insight: one commitment, two arguments. The key design point is to commit the vector $\boldsymbol{a}$ only once as the single commitment C_f, and then reuse the commitment in both permutation and consistency arguments. In the permutation argument, C_f serves as a KZG polynomial commitment to the interpolant of $\boldsymbol{a}$ over H. In the consistency argument, the very same commitment is viewed as a vector commitment to the coordinates of $\boldsymbol{a}$ under the Lagrange basis induced by the SRS. This reuse directly links the permutation and consistency arguments through a single commitment to $\boldsymbol{a}$.

Permutation argument. The goal is to prove that the committed values $a_1, \ldots, a_N$ are exactly a reordering of $(x^1, \ldots, x^N)$. Equivalently, the multisets $\{a_i\}_{i=1}^N$ are identical to $\{x^i\}_{i=1}^N$. The verifier therefore samples a random challenge $\alpha \in \mathbb{F}$ and checks whether $\prod_{i=1}^N (\alpha - a_i) = \prod_{i=1}^N (\alpha - x^i)$. If the multisets differ, this equality holds only with negligible probability over the choice of α. A standard grand-product accumulator (see, e.g., [19]) captures this check via a polynomial $z(X)$ whose evaluations on H satisfy

$$z(\omega^i) = \prod_{j=1}^{i} \tfrac{\alpha - a_j}{\alpha - x^j}, \qquad i \in [N].$$

If the permutation is correct, then the evaluations of z satisfy the required boundary conditions, and in particular the final product equals 1. They also satisfy the multiplicative recurrence between consecutive points in H. Together, these conditions define a low-degree constraint polynomial that is zero on all of H. Hence it is divisible by the vanishing polynomial $Z_H(X) = X^N - 1$. The prover commits to $z(X)$ and the corresponding quotient polynomial, and the verifier checks this divisibility condition using standard batched KZG openings. This yields an $O(1)$-size proof.

Consistency (shuffle correctness) argument. The permutation argument shows that the committed vector $\boldsymbol{a}$ is a permutation of $(x^1, \ldots, x^N)$, but it does not yet show that this same vector correctly relates the input and output ciphertexts. Writing each ciphertext componentwise as $\mathsf{C}_i = (E_i, F_i)$ and $\tilde{\mathsf{C}}_i = (\tilde{E}_i, \tilde{F}_i)$, the required shuffle relation decomposes into one condition for the left component and one for the right. Accordingly, define $V_\ell := \prod_i E_i^{x^i}$ and $V_r := \prod_i F_i^{x^i}$, and write $\boldsymbol{h}_1 = (\tilde{E}_1, \ldots, \tilde{E}_N)$, $\boldsymbol{h}_2 = (\tilde{F}_1, \ldots, \tilde{F}_N)$. We also use the SRS basis $\boldsymbol{g} = (g^{\mathcal{L}_1(\tau)}, \ldots, g^{\mathcal{L}_N(\tau)})$.

Expanding (2) using the ElGamal homomorphism, the shuffle condition reduces to the following three relations:

$$V_\ell = \boldsymbol{h}_1^{\boldsymbol{a}} \cdot g^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle}, \qquad V_r = \boldsymbol{h}_2^{\boldsymbol{a}} \cdot Y^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle}, \qquad C_f = \boldsymbol{g}^{\boldsymbol{a}} \cdot \mathsf{u}_1^{r_1} \mathsf{u}_2^{r_2}, \tag{3}$$

where $\mathsf{u}_1, \mathsf{u}_2 \in \mathbb{G}_1$ are independent generators from the SRS and $r_1, r_2 \in \mathbb{F}$ are random blinding scalars. The first two relations express ciphertext consistency, while the third shows that the same witness vector $\boldsymbol{a}$ is the one committed in C_f. We prove these three relations in three steps:

Step 1: Committing to $\boldsymbol{\rho}$. The prover sends $D := \boldsymbol{k}^{\boldsymbol{\rho}} \cdot g^{\zeta}$, a perfectly hiding Pedersen commitment to $\boldsymbol{\rho}$ using an independent generator vector $\boldsymbol{k}$ and a random blinding scalar ζ.

Step 2: Aggregating the three relations. The verifier samples a random challenge w, and the three equations in (3) are folded into the single relation

$$V_{\text{agg}} = \boldsymbol{h}_{\text{agg}}^{\boldsymbol{a}} \cdot U_{\text{agg}}^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle} \cdot (\mathsf{u}_1^{r_1} \mathsf{u}_2^{r_2})^{w^2}, \tag{4}$$

where $V_{\text{agg}} := V_\ell \cdot V_r^w \cdot C_f^{w^2}$, $\boldsymbol{h}_{\text{agg}} := \boldsymbol{h}_1 \circ \boldsymbol{h}_2^w \circ \boldsymbol{g}^{w^2}$, and $U_{\text{agg}} := g \cdot Y^w$ are derived from public inputs.

Step 3: Reduction to an inner product argument. The aggregated relation (4) reduces the remaining task to proving knowledge of witness vectors $(\boldsymbol{a}, \boldsymbol{\rho})$ satisfying a single inner-product relation. A natural next step is to apply the IPA [9]. However, the standard IPA is <u>not</u> zero-knowledge.

To achieve zero knowledge, the prover masks both witness vectors before invoking the IPA. Concretely, the prover samples two random vectors $\boldsymbol{s}_a, \boldsymbol{s}_\rho \xleftarrow{\$} \mathbb{F}^N$ and defines $\boldsymbol{l}(X) := \boldsymbol{a} + \boldsymbol{s}_a X, \boldsymbol{r}(X) := -\boldsymbol{\rho} + \boldsymbol{s}_\rho X$. Their inner product $t(X) := \langle \boldsymbol{l}(X), \boldsymbol{r}(X) \rangle = t_0 + t_1 X + t_2 X^2$ is a quadratic polynomial in X. Its constant term $t_0 = \langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle$ is precisely the exponent of U_{agg} in (4). The coefficients t_2 and t_1 are committed before the verifier reveals the evaluation challenge u, which fixes the blinded witness vectors $\hat{\boldsymbol{l}} = \boldsymbol{a} + \boldsymbol{s}_a u, \hat{\boldsymbol{r}} = -\boldsymbol{\rho} + \boldsymbol{s}_\rho u$. Using the corresponding openings of the masking commitments, the verifier can then reconstruct a single group element P satisfying the standard IPA relation

$$P = \boldsymbol{h}_{\text{agg}}^{\hat{\boldsymbol{l}}} \cdot \boldsymbol{k}^{\hat{\boldsymbol{r}}} \cdot U_{\text{agg}}^{\langle \hat{\boldsymbol{l}}, \hat{\boldsymbol{r}} \rangle}, \tag{5}$$

Rather than revealing the $2N$ field elements in $\hat{\boldsymbol{l}}$ and $\hat{\boldsymbol{r}}$, the prover and verifier run the inner product argument Π_{IPA} [9], which proves (5) in $\log N$ rounds using only $O(\log N)$ communication.

Putting the two parts together. The shared commitment C_f links the two arguments: it is shown to open to a permutation of $(x^1, \ldots, x^N)$, and the same opening is shown to satisfy the aggregated ciphertext relation with the corresponding rerandomization factors. Hence, the output ciphertexts are a rerandomized permutation of the inputs.

2 Related Work

The study of verifiable shuffles originates from two paradigms. We survey each in turn, progressing from linear-size to sub-linear and logarithmic-size schemes.

Permutation-matrix paradigm (linear size). The first paradigm, introduced by Furukawa and Sako [18], has the prover commit to a permutation matrix and use Schwartz–Zippel-style checks to show that it is consistent with the shuffle. Furukawa et al. [17] gave an implementation for large-scale elections, and Furukawa [15,16] later extended the idea to combine shuffle-and-decryption proofs and aggregate arguments for multi-server mix-nets. Wikström [37] decomposed the argument into offline and online phases, which Terelius and Wikström [36] simplified into a compact argument based on polynomial evaluation over restricted challenge spaces. This has been deployed in real-world elections through the Verificatum library [38] and further optimised by Krips and Lipmaa [30]. The permutation-matrix approach also underlies a line of pairing-based NIZK shuffle arguments in the common reference string model [26,31,11,13,12,2], which progressively improved efficiency and soundness guarantees. Recently, Abdolmaleki et al. [1] proposed shuffle arguments based on subset-checking techniques with improved verifier efficiency. All schemes in this line have $O(N)$ proof size.

Polynomial-roots paradigm and sub-linear schemes. The second paradigm, due to Neff [32], exploits the invariance of polynomials under permutation of their roots. Groth [20,22] refined this with Pedersen commitments and multi-exponentiation techniques. Building on ideas from both paradigms, Groth and Ishai [24] achieved the first sub-linear shuffle argument by arranging the N ciphertexts in an $m \times n$ matrix, achieving $O(m^2 + n)$ communication at the cost of an $O(Nm)$ prover. Bayer and Groth [6] significantly improved this by bringing the communication down to $O(\sqrt{N})$ with an $O(N \log N)$ prover, combining Groth's sub-linear arguments for linear algebra [21] with techniques for computing diagonal sums in the exponent.

Logarithmic-size schemes. Hoffmann, Klooß, and Rupp [28] obtained the first verifiable shuffle with logarithmic proof size by combining the Bayer–Groth framework with inner product arguments [7,9]. Their scheme achieves $O(\log N)$ proof size, $O(N)$ prover complexity, and $O(N)$ verifier complexity, in the plain discrete-log setting with a transparent setup.

3 Preliminaries

3.1 Notation

Let λ be the security parameter. The complexity of algorithms is computed as a function of λ. Let $\mathsf{negl}(\lambda)$ denote an arbitrary negligible function. We write $\mathbb{F}$ for a finite field of prime order p (i.e., $\mathbb{F} = \mathbb{F}_p$). We denote by $\mathbb{F}^{<d}[X]$ the set of univariate polynomials over $\mathbb{F}$ of degree less than d, and by $\mathbb{F}^{\leq d}[X]$ those of degree at most d. For $n \in \mathbb{N}$, let $[n] := \{1, 2, \ldots, n\}$. For a finite set S, we write $s \xleftarrow{\$} S$ to mean that s is sampled uniformly at random from S.

Vectors are typeset in bold. For a vector $\boldsymbol{f}$, we denote its i-th coordinate by f_i. We use $\circ$ to denote the Hadamard (entry-wise) product in the group, i.e.,

$(\boldsymbol{u} \circ \boldsymbol{v})_i = u_i v_i$. Group operations in $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are written multiplicatively with $\cdot$ denoting the group operation. For a group vector $\boldsymbol{u} = (u_1, \ldots, u_n)$ and a scalar $a \in \mathbb{F}$, we write $\boldsymbol{u}^a := (u_1^a, \ldots, u_n^a)$ for component-wise exponentiation. For a field vector $\boldsymbol{a} = (a_1, \ldots, a_n)$, we write $\boldsymbol{u}^{\boldsymbol{a}} := \prod_{i=1}^n u_i^{a_i}$.

Bilinear groups. We work with bilinear groups $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g, h)$ where $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are cyclic groups of prime order p, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a non-degenerate bilinear pairing. The elements $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$ are generators. We assume the existence of efficient algorithms for group operations, pairing evaluation, membership testing, and equality testing.

3.2 Assumptions

Definition 1 (d-DLOG assumption). *Fix a positive integer d. The d-DLOG assumption for $(\mathbb{G}_1, \mathbb{G}_2)$ states that given*

$$[1]_1,\ [x]_1,\ \ldots,\ [x^d]_1, \quad [1]_2,\ [x]_2,\ \ldots,\ [x^d]_2$$

for uniformly chosen $x \xleftarrow{\$} \mathbb{F}$, the probability that any efficient adversary $\mathcal{A}$ outputs x is $\mathsf{negl}(\lambda)$.

Lemma 1 (Schwartz–Zippel [33]). *Let $f \in \mathbb{F}[X]$ be a non-zero polynomial of degree $d \geq 0$. Let S be a finite subset of $\mathbb{F}$, and let x be selected uniformly at random from S. Then $\Pr[f(x) = 0] \leq d/|S|$.*

Lemma 2 (Blinding [34]). *Let $S \subset \mathbb{F}$ with $|S| = n$. Fix a polynomial $f \in \mathbb{F}[X]$. Define the blinded polynomial $\tilde{f}(X) := f(X) + Z_S(X)\rho(X)$, where $\rho(X) \in \mathbb{F}^{<k}[X]$ is a uniformly random polynomial of degree less than k. Then, for any k distinct points $x_1, \ldots, x_k \in \mathbb{F} \setminus S$, the vector $(\tilde{f}(x_1), \ldots, \tilde{f}(x_k))$ is uniformly distributed in $\mathbb{F}^k$.*

3.3 Commitments

We use the standard notions of perfect hiding and computational binding for commitment schemes; see, e.g., [25]. In particular, we use Pedersen commitments over a cyclic group $\mathbb{G}_1$ of prime order p.

Definition 2 (Pedersen vector commitment). *Let $g_1, \ldots, g_n, h \in \mathbb{G}_1$ be public generators. To commit to a vector $\boldsymbol{m} = (m_1, \ldots, m_n) \in \mathbb{F}^n$ with randomness $r \in \mathbb{F}$, compute $\mathsf{Com}(\boldsymbol{m}; r) := h^r \cdot \prod_{i=1}^n g_i^{m_i}$. For $\boldsymbol{m}, \boldsymbol{m}' \in \mathbb{F}^n$ and $r, r' \in \mathbb{F}$, the scheme satisfies*

$$\mathsf{Com}(\boldsymbol{m}; r) \cdot \mathsf{Com}(\boldsymbol{m}'; r') = \mathsf{Com}(\boldsymbol{m} + \boldsymbol{m}';\ r + r').$$

Theorem 1. *The Pedersen vector commitment scheme in Definition 2, including its scalar special case $n = 1$, is perfectly hiding and computationally binding under the discrete logarithm assumption in $\mathbb{G}_1$.*

3.4 ElGamal Encryption

We employ the ElGamal cryptosystem in the group $\mathbb{G}_1$ of prime order p with generator g.

- $\mathsf{KeyGen}(1^\lambda)$: sample $x \xleftarrow{\$} \mathbb{F}$, set $sk = x$ and $pk = Y = g^x$.
- $\mathsf{Enc}_{pk}(M;\, \rho)$: for $M \in \mathbb{G}_1$ and $\rho \xleftarrow{\$} \mathbb{F}$, output $\mathsf{C} = (E, F) = (g^\rho,\, Y^\rho M) \in \mathbb{G}_1^2$.
- $\mathsf{Dec}_{sk}(E, F)$: compute $M = F \cdot E^{-x}$.

ElGamal is multiplicatively homomorphic under component-wise ciphertext multiplication: for any $M_1, M_2 \in \mathbb{G}_1$ and $\rho_1, \rho_2 \in \mathbb{F}$,

$$\mathsf{Enc}_{pk}(M_1;\, \rho_1) \cdot \mathsf{Enc}_{pk}(M_2;\, \rho_2) = \mathsf{Enc}_{pk}(M_1 \cdot M_2;\, \rho_1 + \rho_2).$$

For ciphertext vectors $\mathbf{C} = (\mathsf{C}_1, \ldots, \mathsf{C}_n)$ and scalar vectors $\boldsymbol{a} = (a_1, \ldots, a_n)$:

$$\mathbf{C}^{\boldsymbol{a}} = \prod_{i=1}^{n} \mathsf{C}_i^{a_i} = \mathsf{Enc}_{pk}\Big(\prod_{i=1}^{n} M_i^{a_i};\, \sum_{i=1}^{n} a_i \rho_i\Big).$$

3.5 Interactive Arguments

We adopt the standard conventions and definitions for interactive arguments as presented in [5]. An interactive argument system for a relation $\mathcal{R}$ consists of three PPT algorithms $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$:

- $\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda)$: outputs a common reference string ck.
- $\mathcal{P}(\mathsf{ck}, x, w)$ and $\mathcal{V}(\mathsf{ck}, x)$: the prover and verifier interact; $\mathcal{P}$ holds the witness w while $\mathcal{V}$ does not.

The transcript is denoted $\mathrm{tr} \leftarrow \langle \mathcal{P}(\mathsf{ck}, x, w),\, \mathcal{V}(\mathsf{ck}, x) \rangle$. We write $\langle \mathcal{P}, \mathcal{V} \rangle = 1$ if the verifier accepts. Let $\mathcal{R} \subseteq \{0,1\}^* \times \{0,1\}^* \times \{0,1\}^*$ be a polynomial-time-decidable ternary relation. For a ck, we define $\mathcal{L}_{\mathsf{ck}} = \{x \mid \exists\, w : (\mathsf{ck}; x; w) \in \mathcal{R}\}$.

Definition 3 (Completeness). *An argument system* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *for* $\mathcal{R}$ *has completeness if for all non-uniform PPT adversaries* $\mathcal{A}$*:*

$$\Pr\left[\begin{array}{r|l} (\mathsf{ck}; x; w) \notin \mathcal{R} \vee & \mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda), \\ \langle \mathcal{P}(\mathsf{ck}, x, w),\, \mathcal{V}(\mathsf{ck}, x) \rangle = 1 & (x, w) \leftarrow \mathcal{A}(\mathsf{ck}) \end{array}\right] \geq 1 - \epsilon_c(\lambda).$$

We say the system has perfect completeness if $\epsilon_c(\lambda) = 0$*, and statistical completeness if* ϵ_c *is negligible.*

Definition 4 (Knowledge soundness). *An argument system* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *is knowledge sound for* $\mathcal{R}$ *if for every non-uniform PPT adversary* $\mathcal{A}$*, there exists a PPT extractor* $\mathcal{E}$ *such that:*

$$\Pr\left[\begin{array}{r|l} \langle \mathcal{P}^*, \mathcal{V} \rangle = 1 \wedge & \mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda), \\ (\mathsf{ck}; x; w) \notin \mathcal{R} & ((x, \pi);\, w) \leftarrow (\mathcal{A} \| \mathcal{E})(\mathsf{ck}) \end{array}\right] \leq \mathsf{negl}(\lambda),$$

where $(\mathcal{A} \| \mathcal{E})$ *denotes that* $\mathcal{E}$ *runs with the same random tape as* $\mathcal{A}$ *and may use rewinding.*

Definition 5 (Witness-extended emulation [24, Definition 3]). *A public-coin argument* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *has witness-extended emulation if for all deterministic polynomial-time provers* $\mathcal{P}^*$*, there exists an expected polynomial-time emulator* $\mathcal{E}$ *such that for all non-uniform polynomial-time adversaries* $\mathcal{A}$*:*

$$\Pr\begin{bmatrix}\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda); \\ (x, s) \leftarrow \mathcal{A}(\mathsf{ck}) \\ \mathrm{tr} \leftarrow \langle \mathcal{P}^*(\mathsf{ck}, x, s),\ \mathcal{V}(\mathsf{ck}, x)\rangle : \\ \mathcal{A}(\mathrm{tr}) = 1\end{bmatrix} \approx \Pr\begin{bmatrix}\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda);\ (x, s) \leftarrow \mathcal{A}(\mathsf{ck}) \\ (\mathrm{tr}, w) \leftarrow \mathcal{E}^{\langle \mathcal{P}^*(\mathsf{ck}, x, s),\ \mathcal{V}(\mathsf{ck}, x)\rangle}(\mathsf{ck}, x) : \\ \mathcal{A}(\mathrm{tr}) = 1 \wedge \\ (\textit{if } \mathrm{tr} \textit{ accepting} \Rightarrow (\mathsf{ck}; x; w) \in \mathcal{R})\end{bmatrix}.$$

The emulator $\mathcal{E}$ *has oracle access to the interaction and may rewind* $\mathcal{P}^*$*.*

Definition 6 (Public coin). *An argument system is public coin if all verifier messages are chosen uniformly at random and independently of the prover's messages.*

Definition 7 (Special honest-verifier zero-knowledge). *A public-coin argument* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *for* $\mathcal{R}$ *is special honest-verifier zero-knowledge with simulation error* $\delta(\lambda)$ *if there exists a PPT simulator* $\mathcal{S}$ *such that for every non-uniform PPT adversary* $\mathcal{A}$*,*

$$\left|\Pr\begin{bmatrix}\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, w, \rho) \leftarrow \mathcal{A}(\mathsf{ck}) \\ \mathrm{tr} \leftarrow \langle \mathcal{P}(\mathsf{ck}, x, w), \mathcal{V}(\mathsf{ck}, x; \rho)\rangle : \\ (\mathsf{ck}; x; w) \in \mathcal{R} \wedge \mathcal{A}(\mathrm{tr}) = 1\end{bmatrix} - \Pr\begin{bmatrix}\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, w, \rho) \leftarrow \mathcal{A}(\mathsf{ck}) \\ \mathrm{tr} \leftarrow \mathcal{S}(\mathsf{ck}, x, \rho) : \\ (\mathsf{ck}; x; w) \in \mathcal{R} \wedge \mathcal{A}(\mathrm{tr}) = 1\end{bmatrix}\right| \leq \delta(\lambda).$$

Definition 8 (Trapdoor simulation). *A public-coin argument* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *for* $\mathcal{R}$ *admits trapdoor simulation with error* $\delta(\lambda)$ *if there exist PPT algorithms* $(\mathsf{SimSetup}, \mathcal{S})$ *such that* $\mathsf{SimSetup}(1^\lambda)$ *outputs* (ck, τ) *and, for every* (x, w) *with* $(\mathsf{ck}; x; w) \in \mathcal{R}$ *and every verifier randomness* ρ*, the simulated transcript* $\mathcal{S}(\mathsf{ck}, \tau, x, \rho)$ *is within statistical distance* $\delta(\lambda)$ *of the honest transcript* $\langle \mathcal{P}(\mathsf{ck}, x, w), \mathcal{V}(\mathsf{ck}, x; \rho)\rangle$*, where* $\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda)$ *in the real experiment and* $(\mathsf{ck}, \tau) \leftarrow \mathsf{SimSetup}(1^\lambda)$ *in the simulated experiment.*

3.6 Algebraic Group Model

We analyze security in the Algebraic Group Model (AGM) [14], which lies between the standard model and the generic group model.

We work in the Algebraic Group Model (AGM) [14], where an adversary outputting a group element also provides its algebraic representation with respect to previously seen group elements. This lets us interpret successful pairing checks as corresponding polynomial identities.

Definition 9 (Algebraic adversary [14, Definition 2]). *Let* $\mathbb{G}$ *be a cyclic group of prime order* p*. An algorithm* $\mathcal{A}$ *is algebraic if, whenever* $\mathcal{A}$ *outputs a group element* $Z \in \mathbb{G}$*, it also outputs a representation* $\boldsymbol{v} = (v_1, \ldots, v_k) \in \mathbb{F}^k$ *such that* $Z = \prod_{i=1}^k L_i^{v_i}$*, where* $\boldsymbol{L} = (L_1, \ldots, L_k)$ *is the list of all group elements* $\mathcal{A}$ *has received so far, including the SRS and protocol messages.*

In our context, this means that whenever the adversary outputs a commitment or proof element in $\mathbb{G}_1$, it effectively reveals the polynomial coefficients used to construct that element relative to the SRS.

We say an SRS has degree d if every element in $\mathbb{G}_1$ (resp. $\mathbb{G}_2$) is of the form $g^{f(\tau)}$ (resp. $h^{f'(\tau)}$) for some $f, f' \in \mathbb{F}^{\leq d}[X]$, where τ is the SRS trapdoor.

Lemma 3 (Real vs. ideal checks [19, Lemma 4.5]). *Let $\mathcal{A}$ be an algebraic adversary interacting with a protocol whose SRS has degree d. Suppose $\mathcal{A}$ outputs group elements that, together with SRS elements, are used in a pairing check of the form*

$$\prod_{i=1}^{k} e(A_i,\, B_i) = 1_{\mathbb{G}_T},$$

where each $A_i \in \mathbb{G}_1$ and $B_i \in \mathbb{G}_2$. Let $a_i(X), b_i(X) \in \mathbb{F}[X]$ be the polynomials corresponding to these elements, i.e., $A_i = g^{a_i(\tau)}$ and $B_i = h^{b_i(\tau)}$.

Under the d-DLOG assumption, the probability that $\mathcal{A}$ satisfies the pairing check while the corresponding polynomial identity

$$\sum_{i=1}^{k} a_i(X) b_i(X) \equiv 0$$

does not hold is negligible in λ.

Definition 10 (Knowledge soundness in the AGM). *A protocol Π for a relation $\mathcal{R}$ has knowledge soundness in the AGM if there exists a PPT extractor $\mathcal{E}$ such that for every algebraic adversary $\mathcal{A}$: if $\mathcal{A}$ interacts with the honest verifier and produces an accepting transcript with probability ε, then $\mathcal{E}$, given access to $\mathcal{A}$'s algebraic representations, outputs a witness w with $(x, w) \in \mathcal{R}$ with probability at least $\varepsilon - \mathsf{negl}(\lambda)$.*

3.7 KZG Polynomial Commitment Scheme

We use the KZG polynomial commitment scheme [29].

Definition 11 (KZG polynomial commitment scheme [29]). *For a maximum polynomial degree d, the scheme consists of:*

- $\mathsf{KZG.Setup}(1^\lambda, d)$*: sample a trapdoor $\tau \xleftarrow{\$} \mathbb{F}$ and output the structured reference string* $\mathsf{srs} = \left(\{g^{\tau^i}\}_{i=0}^{d},\ h,\ h^{\tau},\ h^{\tau^2}\right) \in \mathbb{G}_1^{d+1} \times \mathbb{G}_2^3$.
- $\mathsf{KZG.Com}(\mathsf{srs}, f(X))$*: for $f(X) \in \mathbb{F}^{\leq d}[X]$, output $C_f := g^{f(\tau)} \in \mathbb{G}_1$.*

KZG commitments are additively homomorphic in the exponent: $\mathsf{KZG.Com}(f) \cdot \mathsf{KZG.Com}(g) = \mathsf{KZG.Com}(f + g)$.

Theorem 2 (Knowledge soundness of KZG in the AGM [29,19]). *Under the d-DLOG assumption, for any algebraic adversary $\mathcal{A}$ that outputs a KZG commitment C, the polynomial $f(X)$ satisfying $C = g^{f(\tau)}$ can be read directly from $\mathcal{A}$'s algebraic representation. In particular, the committed polynomial is uniquely determined except with negligible probability.*

3.8 Inner Product Argument

We use the inner product argument of Bootle et al. [7] as refined by Bünz et al. [9]. The argument allows a prover to convince a verifier that a committed element P is of the form $\boldsymbol{G}^{\boldsymbol{l}} \cdot \boldsymbol{H}^{\boldsymbol{r}} \cdot U^{\langle \boldsymbol{l}, \boldsymbol{r} \rangle}$ for hidden vectors $\boldsymbol{l}, \boldsymbol{r}$.

Definition 12 (Inner product relation). *Let $\boldsymbol{G}, \boldsymbol{H} \in \mathbb{G}_1^N$ be generator vectors, $U \in \mathbb{G}_1$ a generator for the inner product, and $P \in \mathbb{G}_1$. The inner product relation is $\mathcal{R}_{\mathrm{IPA}} := \big\{ (P, \boldsymbol{G}, \boldsymbol{H}, U;\ \boldsymbol{l}, \boldsymbol{r}) \ :\ \boldsymbol{l}, \boldsymbol{r} \in \mathbb{F}^N,\ P = \boldsymbol{G}^{\boldsymbol{l}} \cdot \boldsymbol{H}^{\boldsymbol{r}} \cdot U^{\langle \boldsymbol{l}, \boldsymbol{r} \rangle} \big\}$.*

We use the asymmetric folding variant of [28], which avoids inverse exponentiations in the recursive step. Let $m := \log N$. We present the protocol in Figure 1.

Protocol Π_{IPA}

1. Initialize $\boldsymbol{G}_0 := \boldsymbol{G}$, $\boldsymbol{H}_0 := \boldsymbol{H}$, $\boldsymbol{l}_0 := \boldsymbol{l}$, $\boldsymbol{r}_0 := \boldsymbol{r}$, $P_0 := P$.

2. For $j = 1$ to m:

(a) Parse $\boldsymbol{l}_{j-1} = (\boldsymbol{l}_L \| \boldsymbol{l}_R)$, $\boldsymbol{r}_{j-1} = (\boldsymbol{r}_L \| \boldsymbol{r}_R)$, $\boldsymbol{G}_{j-1} = (\boldsymbol{G}_L \| \boldsymbol{G}_R)$, $\boldsymbol{H}_{j-1} = (\boldsymbol{H}_L \| \boldsymbol{H}_R)$, each split into halves of size $N/2^j$.

(b) $\mathcal{P} \to \mathcal{V}$: compute $c_L := \langle \boldsymbol{l}_L, \boldsymbol{r}_R \rangle$, $c_R := \langle \boldsymbol{l}_R, \boldsymbol{r}_L \rangle$, and send $L_j := \boldsymbol{G}_R^{\boldsymbol{l}_L} \cdot \boldsymbol{H}_L^{\boldsymbol{r}_R} \cdot U^{c_L}, R_j := \boldsymbol{G}_L^{\boldsymbol{l}_R} \cdot \boldsymbol{H}_R^{\boldsymbol{r}_L} \cdot U^{c_R}$.

(c) $\mathcal{V} \to \mathcal{P}$: sample $\gamma_j \xleftarrow{\$} \mathbb{F}$.

(d) Both parties fold: $\boldsymbol{G}_j := \boldsymbol{G}_L \cdot \boldsymbol{G}_R^{\gamma_j}$, $\boldsymbol{H}_j := \boldsymbol{H}_L^{\gamma_j} \cdot \boldsymbol{H}_R$, $P_j := L_j^{\gamma_j^2} \cdot P_{j-1}^{\gamma_j} \cdot R_j$. $\mathcal{P}$ folds: $\boldsymbol{l}_j := \gamma_j \boldsymbol{l}_L + \boldsymbol{l}_R$, $\boldsymbol{r}_j := \boldsymbol{r}_L + \gamma_j \boldsymbol{r}_R$.

3. $\mathcal{P} \to \mathcal{V}$: send final scalars $\hat{l} := \boldsymbol{l}_m \in \mathbb{F}$ and $\hat{r} := \boldsymbol{r}_m \in \mathbb{F}$.

4. $\mathcal{V}$ computes challenge-product vectors $\boldsymbol{s}, \boldsymbol{s}' \in \mathbb{F}^N$ with $s_i := \prod_{j=1}^m \gamma_j^{b(i,j)}$, $s'_i := \prod_{j=1}^m \gamma_j^{1-b(i,j)}$, where $b(i,j) \in \{0,1\}$ is given by $i - 1 = \sum_{j=1}^m b(i,j)\, 2^{m-j}$, and accepts iff

$$\boldsymbol{G}^{\boldsymbol{s}\,\hat{l}} \cdot \boldsymbol{H}^{\boldsymbol{s}'\hat{r}} \cdot U^{\hat{l}\hat{r}} = P^{\eta_1} \cdot \prod_{j=1}^m \big(L_j^{\gamma_j^2\, \eta_{j+1}} \cdot R_j^{\eta_{j+1}} \big),$$

where $\eta_j := \prod_{k=j}^m \gamma_k$ for $j \in [m]$ and $\eta_{m+1} := 1$.

Fig. 1: Protocol Π_{IPA}.

Theorem 3 ([7,9,28]). *Protocol Π_{IPA} is a $(2m+1)$-move public-coin argument for $\mathcal{R}_{\mathrm{IPA}}$ with perfect completeness and witness-extended emulation under the discrete logarithm assumption in $\mathbb{G}_1$, for any deterministic polynomial-time prover. The proof transcript consists of $2m$ group elements $(L_1, R_1, \ldots, L_m, R_m)$ and 2 field elements $(\hat{l}, \hat{r})$, where $m = \log N$.*

4 Verifiable Shuffle Argument

Let N be the number of ciphertexts to be shuffled. The public input consists of ciphertext vectors $\mathbf{C} := (\mathsf{C}_1, \ldots, \mathsf{C}_N)$ and $\tilde{\mathbf{C}} := (\tilde{\mathsf{C}}_1, \ldots, \tilde{\mathsf{C}}_N)$, where each $\mathsf{C}_i =$

$(E_i, F_i) \in \mathbb{G}_1^2$ is an ElGamal ciphertext. The prover holds a secret permutation $\pi \in \Sigma_N$ and rerandomization factors $\boldsymbol{\rho} \in \mathbb{F}^N$ such that $\tilde{\mathsf{C}}_i = \mathsf{C}_{\pi(i)} \cdot \mathsf{Enc}_{pk}(1; \rho_i)$ for all i. Accordingly, the shuffle relation is

$$\mathcal{R}_{\mathsf{vs}} := \Big\{ (\mathsf{ck}; \mathbf{C}, \tilde{\mathbf{C}}; \pi, \boldsymbol{\rho}) : \forall\, i \in [N],\ \tilde{\mathsf{C}}_i = \mathsf{C}_{\pi(i)} \cdot \mathsf{Enc}_{pk}(1; \rho_i) \Big\}.$$

The protocol operates over a multiplicative subgroup $H = \{\omega, \omega^2, \ldots, \omega^N\} \subset \mathbb{F}$ of order $N = 2^\mu$, with $\mathcal{L}_i(X)$ the i-th Lagrange basis polynomial over H and $Z_H(X) = \prod_{i=1}^N (X - \omega^i)$ the vanishing polynomial.

Structured reference string. The protocol uses an updatable powers-of-tau SRS supporting commitments to polynomials of degree at most $N+3$. Let $\tau \in \mathbb{F} \setminus H$ denote the secret evaluation point. The commitment key is

$$\mathsf{ck} = \Big(H \subset \mathbb{F}, \{g^{\tau^i}\}_{i=0}^{N+3} \in \mathbb{G}_1^{N+4}, \boldsymbol{g}, \boldsymbol{k} \in \mathbb{G}_1^N, g, Y, \mathsf{u}_1, \mathsf{u}_2, \mathsf{u}_3 \in \mathbb{G}_1, h, h^\tau, h^{\tau^2} \in \mathbb{G}_2 \Big),$$

where $g_i = g^{\mathcal{L}_i(\tau)}$ for $i \in [N]$, and $\mathsf{u}_j = g^{\tau^{j-1} Z_H(\tau)}$ for $j \in \{1, 2, 3\}$. The vector $\boldsymbol{k}$ consists of independently sampled generators.

The monomial SRS allows the prover to commit to any polynomial of degree at most $N+3$ as $g^{f(\tau)}$. The Lagrange-basis elements $\boldsymbol{g}$ and blinding bases $\mathsf{u}_1, \mathsf{u}_2, \mathsf{u}_3$ are listed separately as precomputed conveniences: any polynomial of the form $\sum_i c_i \mathcal{L}_i(X) + (d_2 X^2 + d_1 X + d_0)\, Z_H(X)$ can be committed to directly as $\prod_i g_i^{c_i} \cdot \mathsf{u}_3^{d_2} \cdot \mathsf{u}_2^{d_1} \cdot \mathsf{u}_1^{d_0}$. We also write

$$\boldsymbol{h}_1 := (\tilde{E}_1, \ldots, \tilde{E}_N) \in \mathbb{G}_1^N, \quad \boldsymbol{h}_2 := (\tilde{F}_1, \ldots, \tilde{F}_N) \in \mathbb{G}_1^N$$

for the output ciphertext component vectors.

We now make the outline of Section 1.1 concrete in polynomial form. After the verifier samples $x \xleftarrow{\$} \mathbb{F}$, the prover forms the vector $\boldsymbol{a} = (x^{\pi(1)}, \ldots, x^{\pi(N)})$ and commits to its blinded interpolant $f^*(X) = \sum_{i=1}^N a_i \mathcal{L}_i(X) + (r_1 + r_2 X) Z_H(X)$, where $r_1, r_2 \xleftarrow{\$} \mathbb{F}$.

Let $I(X) := \sum_{i=1}^N x^i \mathcal{L}_i(X)$, which interpolates $(x^1, \ldots, x^N)$ over H. The prover constructs a grand-product accumulator polynomial $z(X)$ satisfying $z(\omega) = 1$, and $z(\omega^{i+1}) = z(\omega^i) \frac{a_i - \alpha}{x^i - \alpha}$ for $i = 1, \ldots, N-1$, where α is a verifier challenge. Accordingly, over the domain H, the permutation check is enforced by the recurrence constraint

$$(f^*(X) - \alpha)\, z(X) = (I(X) - \alpha)\, z(\omega X)$$

together with the boundary condition $(z(X) - 1)\, \mathcal{L}_1(X) = 0$. To batch these constraints, the verifier samples a challenge $\gamma_1 \xleftarrow{\$} \mathbb{F}$, and we define

$$p(X) := (f^*(X) - \alpha)\, z(X) - (I(X) - \alpha)\, z(\omega X) + \gamma_1 (z(X) - 1) \mathcal{L}_1(X).$$

To show that these constraints vanish on H, the prover sends a quotient polynomial $q(X)$ such that $p(X) = Z_H(X)\, q(X)$. The verifier then samples a random

Protocol Π_{vs}

1. $\mathcal{V} \to \mathcal{P}$: sample $x \xleftarrow{\$} \mathbb{F}$.
2. $\mathcal{P} \to \mathcal{V}$: set $\boldsymbol{a} := (x^{\pi(1)}, \ldots, x^{\pi(N)})$, sample $r_1, r_2 \xleftarrow{\$} \mathbb{F}$, define
$$f^*(X) := \textstyle\sum_i a_i\, \mathcal{L}_i(X) + (r_1 + r_2 X)\, Z_H(X),$$
and send $C_f := \prod_{i=1}^N g_i^{a_i} \cdot \mathsf{u}_1^{r_1} \cdot \mathsf{u}_2^{r_2}$.
3. $\mathcal{V} \to \mathcal{P}$: sample $\alpha \xleftarrow{\$} \mathbb{F} \setminus \{x^1, \ldots, x^N\}$.
4. $\mathcal{P} \to \mathcal{V}$: sample $b_1, b_2, b_3 \xleftarrow{\$} \mathbb{F}$. Since Z_H vanishes on H, $f^*(\omega^j) = a_j$ and $I(\omega^j) = x^j$ for all $j \in [N]$. Compute
$$z(X) := (b_1 X^2 + b_2 X + b_3)\, Z_H(X) + \mathcal{L}_1(X) + \textstyle\sum_{i=1}^{N-1} \mathcal{L}_{i+1}(X) \prod_{j=1}^{i} \frac{a_j - \alpha}{x^j - \alpha},$$
and send $C_z := \prod_i g_i^{z(\omega^i)} \cdot \mathsf{u}_3^{b_1} \cdot \mathsf{u}_2^{b_2} \cdot \mathsf{u}_1^{b_3}$.
5. $\mathcal{V} \to \mathcal{P}$: sample $\gamma_1 \xleftarrow{\$} \mathbb{F}$.
6. $\mathcal{P} \to \mathcal{V}$: compute $q(X) := \frac{(f^*(X) - \alpha)\, z(X) - (I(X) - \alpha)\, z(\omega X) + \gamma_1\, (z(X) - 1)\, \mathcal{L}_1(X)}{Z_H(X)}$, and send $C_q := \mathsf{KZG.Com}(q)$.
7. $\mathcal{V} \to \mathcal{P}$: sample the evaluation challenge $v \xleftarrow{\$} \mathbb{F} \setminus H$.
8. $\mathcal{P} \to \mathcal{V}$: compute and send the evaluations $\hat{f} := f^*(v)$, $\hat{z} := z(\omega v)$.
9. $\mathcal{V} \to \mathcal{P}$: sample the batching challenges $\gamma_2, \gamma_3 \xleftarrow{\$} \mathbb{F}$.
10. Let
$$r(X) := (\hat{f} - \alpha)\, z(X) - (I(v) - \alpha)\, \hat{z} + \gamma_1\, (z(X) - 1)\, \mathcal{L}_1(v) - Z_H(v)\, q(X).$$
Compute
$$W(X) := \frac{r(X) + \gamma_2 (f^*(X) - \hat{f})}{X - v} + \frac{\gamma_3 (z(X) - \hat{z})}{X - \omega v}.$$
Send $C_w := \mathsf{KZG.Com}(W)$.
11. **Permutation check.** $\mathcal{V}$ computes $Z_H(v)$, $\mathcal{L}_1(v)$, $I(v)$, and reconstructs
$$C_R := C_z^{\hat{f} - \alpha + \gamma_1 \mathcal{L}_1(v)} \cdot g^{-(I(v) - \alpha)\hat{z} - \gamma_1 \mathcal{L}_1(v)} \cdot C_q^{-Z_H(v)}.$$
Accept iff
$$e(C_w,\, h^{(\tau - v)(\tau - \omega v)}) = e\big(C_R \cdot C_f^{\gamma_2} \cdot g^{-\gamma_2 \hat{f}},\, h^{\tau - \omega v}\big) \cdot e\big(C_z^{\gamma_3} \cdot g^{-\gamma_3 \hat{z}},\, h^{\tau - v}\big).$$
12. **Shuffle correctness.** $\mathcal{V}$ computes $V_\ell := \prod_i E_i^{x^i}$, $V_r := \prod_i F_i^{x^i}$. Both parties run Π_{Shuffle} (Section 5) to prove $(\mathsf{ck}, V_\ell, V_r, C_f, \boldsymbol{h}_1, \boldsymbol{h}_2;\, \boldsymbol{a}, \boldsymbol{\rho}, r_1, r_2) \in \mathcal{R}_{\mathrm{Shuffle}}$.
13. $\mathcal{V}$ outputs acc iff both Steps 11 and 12 accept.

Fig. 2: Protocol Π_{vs}.

point $v \notin H$ and requests the evaluations $\hat{f} := f^*(v), \hat{z} := z(\omega v)$. Substituting these values yields a polynomial linear in the remaining committed polynomials

z and q, namely

$$r(X) := (\hat{f} - \alpha)\, z(X) - (I(v) - \alpha)\, \hat{z} + \gamma_1 (z(X) - 1)\mathcal{L}_1(v) - Z_H(v)\, q(X).$$

If $p(X) = Z_H(X)\, q(X)$, then $r(v) = 0$. Thus, it remains to check $r(v) = 0, f^*(v) = \hat{f}, z(\omega v) = \hat{z}$. These three claims are checked with a single batched KZG opening using fresh challenges γ_2, γ_3.

4.1 Protocol Construction

The prover $\mathcal{P}(\mathsf{ck}, \mathbf{C}, \tilde{\mathbf{C}}, \pi, \boldsymbol{\rho})$ and verifier $\mathcal{V}(\mathsf{ck}, \mathbf{C}, \tilde{\mathbf{C}})$ interact as in Figure 2. Steps 1–11 establish the permutation argument and produce the shared commitment C_f. Step 12 invokes Π_{Shuffle} (Section 5) to prove consistency with the ciphertext shuffle using the same C_f.

Theorem 4. *In the algebraic group model, under the d-DLOG assumption and the discrete logarithm assumption in $\mathbb{G}_1$, Protocol Π_{vs} is a perfectly complete and admits statistical special honest-verifier zero-knowledge with simulation trapdoor for $\mathcal{R}_{\mathsf{vs}}$. Moreover, its permutation sub-argument (Steps 1–11) is knowledge sound in the AGM, and its shuffle-consistency sub-argument (Step 12) has witness-extended emulation under the discrete logarithm assumption. Together, these properties yield extraction of a valid shuffle witness except with negligible probability.*

The proof is deferred to Appendix A.1.

5 Shuffle Correctness Argument

This section specifies the sub-protocol Π_{Shuffle} used in Step 10 of Π_{vs} (Section 4.1). It proves that the commitment C_f produced in the permutation argument is consistent with the shuffled ciphertexts. We define the relation

$$\mathcal{R}_{\text{Shuffle}} := \left\{ \begin{array}{c} \big(\mathsf{ck};\ V_\ell,\, V_r,\, C_f,\, \boldsymbol{h}_1,\, \boldsymbol{h}_2;\ \boldsymbol{a},\, \boldsymbol{\rho},\, r_1,\, r_2\big)\ :\ \boldsymbol{a}, \boldsymbol{\rho} \in \mathbb{F}^N,\ r_1, r_2 \in \mathbb{F}, \\ V_\ell = \boldsymbol{h}_1^{\boldsymbol{a}} \cdot g^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle}, \quad V_r = \boldsymbol{h}_2^{\boldsymbol{a}} \cdot Y^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle}, \quad C_f = \boldsymbol{g}^{\boldsymbol{a}} \cdot \mathsf{u}_1^{r_1} \cdot \mathsf{u}_2^{r_2} \end{array} \right\}.$$

The public inputs are $V_\ell, V_r, C_f, \boldsymbol{h}_1, \boldsymbol{h}_2$, and the witness is $(\boldsymbol{a}, \boldsymbol{\rho}, r_1, r_2)$, where r_1, r_2 are the blinding scalars from Step 2 of Π_{vs}.

As outlined in Section 1.1, the verifier samples a challenge w and uses it to compress the three defining equations of $\mathcal{R}_{\text{Shuffle}}$ into the single aggregated relation $V_{\text{agg}} = \boldsymbol{h}_{\text{agg}}^{\boldsymbol{a}} \cdot U_{\text{agg}}^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle} \cdot (\mathsf{u}_1^{r_1} \mathsf{u}_2^{r_2})^{w^2}$, where V_{agg}, $\boldsymbol{h}_{\text{agg}}$, and U_{agg} are defined in (4). The prover then defines the masked vectors $\hat{\boldsymbol{l}} = \boldsymbol{a} + \boldsymbol{s}_a\, u, \hat{\boldsymbol{r}} = -\boldsymbol{\rho} + \boldsymbol{s}_\rho\, u$, and the resulting statement is reduced to an instance of $\mathcal{R}_{\text{IPA}}$ from Section 3.8.

Protocol Π_{Shuffle}

1. $\mathcal{P} \to \mathcal{V}$: sample $\zeta \xleftarrow{\$} \mathbb{F}$ and send $D := \boldsymbol{k}^{\boldsymbol{\rho}} \cdot g^{\zeta}$.
2. $\mathcal{V} \to \mathcal{P}$: sample $w \xleftarrow{\$} \mathbb{F}$.
3. Both parties compute: $\boldsymbol{h}_{\text{agg}} := \boldsymbol{h}_1 \circ \boldsymbol{h}_2^{w} \circ \boldsymbol{g}^{w^2}$, $U_{\text{agg}} := g \cdot Y^{w}$, $V_{\text{agg}} := V_{\ell} \cdot V_r^{w} \cdot C_f^{w^2}$.
4. $\mathcal{P} \to \mathcal{V}$: sample $\boldsymbol{s}_a, \boldsymbol{s}_{\rho} \xleftarrow{\$} \mathbb{F}^N$ and $(\delta, \rho_{r_1}, \rho_{r_2}, \beta_1, \beta_2) \xleftarrow{\$} \mathbb{F}^5$. Let $\boldsymbol{l}(X) := \boldsymbol{a} + \boldsymbol{s}_a X$ and $\boldsymbol{r}(X) := -\boldsymbol{\rho} + \boldsymbol{s}_{\rho} X$, and define
$$t(X) := \langle \boldsymbol{l}(X), \boldsymbol{r}(X) \rangle = t_0 + t_1 X + t_2 X^2,$$
where $t_0 = \langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle$, $t_1 = \langle \boldsymbol{a}, \boldsymbol{s}_{\rho} \rangle + \langle \boldsymbol{s}_a, -\boldsymbol{\rho} \rangle$, $t_2 = \langle \boldsymbol{s}_a, \boldsymbol{s}_{\rho} \rangle$. Send
$$A := \boldsymbol{h}_{\text{agg}}^{\boldsymbol{s}_a} \cdot \boldsymbol{k}^{\boldsymbol{s}_{\rho}} \cdot \mathsf{u}_1^{\rho_{r_1}} \cdot \mathsf{u}_2^{\rho_{r_2}} \cdot g^{\delta}, \quad T_1 := U_{\text{agg}}^{t_1} \cdot g^{\beta_1}, \quad T_2 := U_{\text{agg}}^{t_2} \cdot g^{\beta_2}.$$
5. $\mathcal{V} \to \mathcal{P}$: sample $u \xleftarrow{\$} \mathbb{F} \setminus \{0\}$.
6. $\mathcal{P} \to \mathcal{V}$: compute $\hat{\boldsymbol{l}} := \boldsymbol{a} + \boldsymbol{s}_a u$, $\hat{\boldsymbol{r}} := -\boldsymbol{\rho} + \boldsymbol{s}_{\rho} u$, and send $\beta_x := \beta_1 u + \beta_2 u^2$, $\mu := \delta u - \zeta$, $\mu_1 := \rho_{r_1} u + r_1 w^2$, $\mu_2 := \rho_{r_2} u + r_2 w^2$.
7. $\mathcal{V}$ computes:
$$P := V_{\text{agg}} \cdot A^{u} \cdot D^{-1} \cdot T_1^{u} \cdot T_2^{u^2} \cdot \mathsf{u}_1^{-\mu_1} \cdot \mathsf{u}_2^{-\mu_2} \cdot g^{-\mu - \beta_x}.$$
8. $\mathcal{P}$ and $\mathcal{V}$ execute Π_{IPA} (Figure 1) on input $(\boldsymbol{h}_{\text{agg}}, \boldsymbol{k}, U_{\text{agg}}, P; \hat{\boldsymbol{l}}, \hat{\boldsymbol{r}}) \in \mathcal{R}_{\text{IPA}}$.
9. $\mathcal{V}$ outputs acc iff Π_{IPA} accepts.

Fig. 3: Protocol Π_{Shuffle}.

5.1 Protocol Construction

The prover $\mathcal{P}(\mathsf{ck}, V_{\ell}, V_r, C_f, \boldsymbol{h}_1, \boldsymbol{h}_2;\ \boldsymbol{a}, \boldsymbol{\rho}, r_1, r_2)$ and verifier $\mathcal{V}(\mathsf{ck}, V_{\ell}, V_r, C_f, \boldsymbol{h}_1, \boldsymbol{h}_2)$ interact as in Figure 3.

Theorem 5. *Under the discrete logarithm assumption in $\mathbb{G}_1$, Protocol Π_{Shuffle} is a perfectly complete, perfect special honest-verifier zero-knowledge argument for $\mathcal{R}_{\text{Shuffle}}$ with witness-extended emulation.*

The proof is deferred to Appendix A.2.

6 Efficiency Analysis and Implementation

6.1 Complexity Analysis

We analyze the proof size, prover computation, and verifier computation of our verifiable shuffle. A comparison with prior schemes appears in Table 1.

Proof Size. The complete proof consists of $(2\log N + 8)$ elements in $\mathbb{G}_1$ and 8 elements in $\mathbb{F}$. Concretely, Π_{vs} contributes four commitments (C_f, C_z, C_q, C_w) and two evaluations $(\hat{f}, \hat{z})$, while Π_{Shuffle} contributes four commitments (D, A, T_1, T_2), four scalars $(\beta_x, \mu, \mu_1, \mu_2)$, $2\log N$ cross-term commitments (L_j, R_j), and two final scalars $(\hat{l}, \hat{r})$. For compressed BLS12-381 encodings (48 bytes for $\mathbb{G}_1$ elements and 32 bytes for field elements), this yields proof sizes ranging from 1.28 KB at $N = 2^7$ to 2.5 KB at $N = 2^{20}$, as reported in Table 2.

Prover Computation. Counting group exponentiations, the prover performs:

- Permutation argument $(4N)$: Computing C_f, C_z, C_q, and C_w costs $4N$.
- Pre-IPA phase of Π_{Shuffle} $(5N)$: Forming $\boldsymbol{h}_{\mathrm{agg}}$ costs $2N$, while the commitments D and A cost N and $2N$, respectively.
- Inner-product argument Π_{IPA} $(6N)$: The cross-term commitments (L_j, R_j) over $m = \log N$ rounds cost $4\sum_{j=1}^{m} N/2^j < 4N$, and generator folding costs $2\sum_{j=1}^{m} N/2^j < 2N$.

In addition to the $15N$ group exponentiations, the prover performs three FFTs over $\mathbb{F}$. These add an $O(N \log N)$ field-arithmetic term. Since field operations are much cheaper than group exponentiations on pairing-friendly curves, this overhead is small in our evaluated range: in our benchmarks, the FFTs account for less than 2% of total prover time across all tested values (Section 6.2).

Verifier Computation. Counting group exponentiations, the verifier performs:

- Aggregation $(4N)$: Computing $V_\ell = \prod_i E_i^{x^i}$, $V_r = \prod_i F_i^{x^i}$, and $\boldsymbol{h}_{\mathrm{agg}}$.
- IPA verification $(2N)$: The final check of [9] collapses all $\log N$ folding rounds into a single $(2N+2\log N+2)$-point MSM via the $\boldsymbol{s}$-vector technique, costing $\approx 2N$ exponentiations.

In addition, the verifier performs three pairing computations in Π_{vs}.

6.2 Implementation and Benchmarks

We implement our protocol in Rust[1], using the `dusk-bls12_381`[2] and `dusk-plonk`[3] libraries for BLS12-381 field arithmetic, group operations, and KZG polynomial commitments. Non-interactivity is achieved via the Fiat–Shamir heuristic, instantiated with the `merlin` transcript library to bind public statements and derive challenges.

All benchmarks are run single-threaded on a consumer laptop (MacBook Pro, 16-inch, November 2023) with an Apple M3 Pro chip, 18 GB unified memory. Table 2 reports prover time, verifier time, and proof size for $N = 2^7$ to $N = 2^{20}$.

[1] Source code available at `https://github.com/yuxi16/verifiable-shuffle`.
[2] `https://crates.io/crates/dusk-bls12-381`
[3] `https://github.com/dusk-network/plonk`

Table 2: Benchmark results and proof-size comparison. The "Prover" and "Verifier" columns report measured single-threaded runtimes for our protocol on BLS12-381 (48 B compressed $\mathbb{G}_1$, 32 B $\mathbb{F}$). The three shaded columns are proof-size estimates computed from the formulas in Table 1, instantiated over Curve25519 (32 B group element, 32 B scalar).

N	Prover	Verifier	**Ours** (KB)	[28] (KB)	[6] (KB)	[36]
2^7	159 ms	13 ms	1.28	1.69	6.75	28.0 KB
2^8	305 ms	20 ms	1.38	1.88	8.00	56.0 KB
2^9	595 ms	25 ms	1.47	2.06	13.5	112 KB
2^{10}	1.17 s	37 ms	1.56	2.25	16.0	224 KB
2^{11}	2.31 s	64 ms	1.66	2.44	27.0	448 KB
2^{12}	4.57 s	101 ms	1.75	2.63	32.0	896 KB
2^{13}	9.14 s	183 ms	1.84	2.81	54.0	1.75 MB
2^{14}	18.2 s	327 ms	1.94	3.00	64.0	3.50 MB
2^{15}	36.3 s	640 ms	2.03	3.19	108	7.00 MB
2^{16}	72.3 s	1.19 s	2.13	3.38	128	14.0 MB
2^{17}	145 s	2.23 s	2.22	3.56	216	28.0 MB
2^{18}	289 s	4.53 s	2.31	3.75	256	56.0 MB
2^{19}	577 s	8.95 s	2.41	3.94	432	112 MB
2^{20}	1157 s	17.8 s	2.50	4.13	512	224 MB

Discussion. The measurements in Table 2 are consistent with the theoretical cost analysis. Prover time grows essentially linearly with N, with an empirical doubling ratio close to 2. Verifier time also approaches linear scaling for larger N, while for smaller instances the constant cost of the pairing checks remains visible. The proof size grows only logarithmically, from 1.28 KB at $N = 2^7$ to 2.5 KB at $N = 2^{20}$.

Prover/verifier gap. In practice, the prover is much slower than the verifier, although the coarse operation counts differ by only a constant factor. The main reason is that the verifier's dominant work is concentrated in a single large MSM, which is highly optimized in existing libraries. By contrast, the prover's work is distributed across multiple protocol stages and, in particular, across the recursive rounds of the IPA, where it is spread over many smaller computations. As a result, the measured prover/verifier gap is substantially larger than what the raw $15N$ versus $6N$ accounting alone would suggest.

References

1. Abdolmaleki, B., Fauzi, P., Krips, T., Siim, J.: Shuffle arguments based on subset-checking. In: International Conference on Security and Cryptography for Networks. pp. 345–366. Springer (2024)
2. Abdolmaleki, B., Lipmaa, H., Siim, J., Zając, M.: A non-interactive shuffle argument with low trust assumptions. In: Cryptographers' Track at the RSA Conference. pp. 667–692. Springer (2020)

3. Aranha, D.F., Baum, C., Gjøsteen, K., Silde, T.: Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 1467–1481 (2023)
4. Aranha, D.F., Baum, C., Gjøsteen, K., Silde, T., Tunge, T.: Lattice-based proof of shuffle and applications to electronic voting. In: Cryptographers' Track at the RSA Conference. pp. 227–251. Springer (2021)
5. Attema, T., Fehr, S., Klooß, M.: Fiat-shamir transformation of multi-round interactive proofs. In: Theory of Cryptography Conference. pp. 113–142. Springer (2022)
6. Bayer, S., Groth, J.: Efficient zero-knowledge argument for correctness of a shuffle. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 263–280. Springer (2012)
7. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 327–357. Springer (2016)
8. Bowe, S., Gabizon, A., Miers, I.: Scalable multi-party computation for zk-snark parameters in the random beacon model. Cryptology ePrint Archive (2017)
9. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE symposium on security and privacy (SP). pp. 315–334. IEEE (2018)
10. Chaum, D.L.: Untraceable electronic mail, return addresses, and digital pseudonyms. Communications of the ACM **24**(2), 84–88 (1981)
11. Fauzi, P., Lipmaa, H.: Efficient culpably sound nizk shuffle argument without random oracles. In: Cryptographers' Track at the RSA Conference. pp. 200–216. Springer (2016)
12. Fauzi, P., Lipmaa, H., Siim, J., Zając, M.: An efficient pairing-based shuffle argument. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 97–127. Springer (2017)
13. Fauzi, P., Lipmaa, H., Zając, M.: A shuffle argument secure in the generic model. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 841–872. Springer (2016)
14. Fuchsbauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: Annual International Cryptology Conference. pp. 33–62. Springer (2018)
15. Furukawa, J.: Efficient, verifiable shuffle decryption and its requirement of unlinkability. In: International Workshop on Public Key Cryptography. pp. 319–332. Springer (2004)
16. Furukawa, J.: Efficient and verifiable shuffling and shuffle-decryption. IEICE transactions on fundamentals of electronics, communications and computer sciences **88**(1), 172–188 (2005)
17. Furukawa, J., Miyauchi, H., Mori, K., Obana, S., Sako, K.: An implementation of a universally verifiable electronic voting scheme based on shuffling. In: International Conference on Financial Cryptography. pp. 16–30. Springer (2002)
18. Furukawa, J., Sako, K.: An efficient scheme for proving a shuffle. In: Annual International Cryptology Conference. pp. 368–387. Springer (2001)
19. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
20. Groth, J.: A verifiable secret shuffle of homomorphic encryptions. In: International Workshop on Public Key Cryptography. pp. 145–160. Springer (2003)

21. Groth, J.: Linear algebra with sub-linear zero-knowledge arguments. In: Annual International Cryptology Conference. pp. 192–208. Springer (2009)
22. Groth, J.: A verifiable secret shuffle of homomorphic encryptions. Journal of Cryptology **23**(4), 546–579 (2010)
23. Groth, J.: On the size of pairing-based non-interactive arguments. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 305–326. Springer (2016)
24. Groth, J., Ishai, Y.: Sub-linear zero-knowledge argument for correctness of a shuffle. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 379–396. Springer (2008)
25. Groth, J., Kohlweiss, M.: One-out-of-many proofs: Or how to leak a secret and spend a coin. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 253–280. Springer (2015)
26. Groth, J., Lu, S.: A non-interactive shuffle with pairing based verifiability. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 51–67. Springer (2007)
27. Haines, T., Goré, R., Tiwari, M.: Machine-checking multi-round proofs of shuffle: Terelius-Wikström and Bayer-Groth. In: USENIX Security (2023)
28. Hoffmann, M., Klooß, M., Rupp, A.: Efficient zero-knowledge arguments in the discrete log setting, revisited. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 2093–2110 (2019)
29. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16. pp. 177–194. Springer (2010)
30. Krips, T., Lipmaa, H.: More efficient shuffle argument from unique factorization. In: Cryptographers' Track at the RSA Conference. pp. 252–275. Springer (2021)
31. Lipmaa, H., Zhang, B.: A more efficient computationally sound non-interactive zero-knowledge shuffle argument. In: International Conference on Security and Cryptography for Networks. pp. 477–502. Springer (2012)
32. Neff, C.A.: A verifiable secret shuffle and its application to e-voting. In: Proceedings of the 8th ACM conference on Computer and Communications Security. pp. 116–125 (2001)
33. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. Journal of the ACM (JACM) **27**(4), 701–717 (1980)
34. Sefranek, M.: How (not) to simulate plonk. In: International Conference on Security and Cryptography for Networks. pp. 96–117. Springer (2024)
35. Seres, I.A., Nagy, D.A., Buckland, C., Burcsi, P.: MixEth: Efficient, trustless coin mixing service for Ethereum. In: International Conference on Blockchain Economics, Security and Protocols (Tokenomics). OASIcs, vol. 71, pp. 13:1–13:20. Schloss Dagstuhl (2020)
36. Terelius, B., Wikström, D.: Proofs of restricted shuffles. In: International Conference on Cryptology in Africa. pp. 100–113. Springer (2010)
37. Wikström, D.: A commitment-consistent proof of a shuffle. In: Australasian Conference on Information Security and Privacy. pp. 407–421. Springer (2009)
38. Wikström, D.: Verificatum mix-net. `http://www.verificatum.org` (2010)

A Security Analysis

A.1 Proof of Theorem 4

Proof. We prove each property in turn.

Perfect completeness. Suppose the prover holds a valid witness $(\pi, \boldsymbol{\rho})$ and follows the protocol honestly. The accumulator recurrence of z is satisfied at every $\omega^i \in H$, the quotient $q = p/Z_H$ is well-defined, and all evaluations are computed correctly. Therefore the pairing equation in Step 11 holds by direct substitution. Step 12 invokes Π_{Shuffle}, whose perfect completeness is established in Theorem 5. Finally, the SRS is generated with $\tau \notin H$ (Section 4), so all SRS elements are well-defined. Thus an honest prover is always accepted.

Statistical special honest-verifier zero-knowledge with simulation trapdoor. We construct a simulator $\mathcal{S}$ that receives the public statement, all verifier challenges, and the SRS trapdoor τ. $\mathcal{S}$ proceeds as follows.

1. Compute $I(\tau)$, $Z_H(\tau)$, and $\mathcal{L}_1(\tau)$.
2. Sample $f_\tau, z_\tau \xleftarrow{\$} \mathbb{F}$ uniformly and set $C_f := g^{f_\tau}$, $C_z := g^{z_\tau}$.
3. Sample $z_{\omega\tau} \xleftarrow{\$} \mathbb{F}$ and define $q_\tau := \frac{(f_\tau - \alpha)\, z_\tau - (I(\tau) - \alpha)\, z_{\omega\tau} + \gamma_1 (z_\tau - 1)\, \mathcal{L}_1(\tau)}{Z_H(\tau)}$. Set $C_q := g^{q_\tau}$.
4. Receive the evaluation challenge v. If $v = \tau$ or $\omega v = \tau$, abort. Otherwise compute $I(v)$, $Z_H(v)$, $\mathcal{L}_1(v)$.
5. Sample $\hat{f}, \hat{z} \xleftarrow{\$} \mathbb{F}$ uniformly.
6. Receive the batching challenges γ_2, γ_3 and define $r_\tau := (\hat{f} - \alpha)\, z_\tau - (I(v) - \alpha)\, \hat{z} + \gamma_1 (z_\tau - 1)\, \mathcal{L}_1(v) - Z_H(v)\, q_\tau$ and $W_\tau := \frac{r_\tau + \gamma_2 (f_\tau - \hat{f})}{\tau - v} + \frac{\gamma_3 (z_\tau - \hat{z})}{\tau - \omega v}$. Set $C_w := g^{W_\tau}$.
7. Invoke the sub-simulator $\mathcal{S}_{\text{Shuffle}}$ from Theorem 5 on the statement (V_ℓ, V_r, C_f, $\boldsymbol{h}_1$, $\boldsymbol{h}_2$) and the verifier challenges of Step 12, obtaining a simulated sub-transcript π_{Shuffle}.
8. Output $(C_f, C_z, C_q, C_w, \hat{f}, \hat{z}, \pi_{\text{Shuffle}})$.

Indistinguishability. In the real protocol, f^* is blinded by $(r_1 + r_2 X) Z_H(X)$ and z by $(b_1 X^2 + b_2 X + b_3) Z_H(X)$. By Lemma 2, provided the evaluation points are pairwise distinct and lie outside H, the tuple $(f^*(\tau), f^*(v))$ is uniformly distributed over $\mathbb{F}^2$, and likewise $(z(\tau), z(\omega\tau), z(\omega v))$ is uniformly distributed over $\mathbb{F}^3$. The simulator samples exactly these values from the same distributions. Once these values and the verifier challenges are fixed, the values q_τ and W_τ are deterministically determined, so the distributions of $(C_f, C_z, C_q, \hat{f}, \hat{z}, C_w)$ in the real and simulated transcripts coincide. The shuffle sub-transcript is identically distributed by Theorem 5. Let E be the event that

$$\tau, v, \omega\tau, \omega v \notin H, \qquad v \neq \tau, \qquad \omega v \neq \tau, \qquad \tau \neq \omega\tau, \qquad \tau \neq \omega v.$$

Since $\tau \notin H$ is fixed by setup and v is sampled uniformly from $\mathbb{F} \setminus H$, all failures in E occur with probability at most $O(1/|\mathbb{F}|)$. Conditioned on E, the simulated and real transcripts are identically distributed. Hence the statistical distance between the two transcript distributions is negligible.

Knowledge soundness. Let $\mathcal{A}$ be an algebraic adversary (Definition 9) that produces an accepting transcript with non-negligible probability ε. We construct an extractor $\mathcal{E}$ that outputs $(\pi^*, \boldsymbol{\rho}^*)$ with $(\mathsf{ck}; \mathbf{C}, \tilde{\mathbf{C}}; \pi^*, \boldsymbol{\rho}^*) \in \mathcal{R}_{\mathsf{vs}}$, except with negligible probability.

Since $\mathcal{A}$ is algebraic, every group element it outputs comes with an algebraic representation relative to the SRS and all previously received protocol messages. In particular, from C_f the extractor reads the polynomial $f^*(X)$ with evaluation vector $(a_1^*, \ldots, a_N^*)$ and blinding scalars (r_1^*, r_2^*), from C_z it reads $z(X)$, from C_q it reads $q(X)$.

By Lemma 3, acceptance of the pairing check in Step 11 implies, under the d-DLOG assumption and except with negligible probability, that the committed polynomial W satisfies

$$\begin{aligned} W(X)(X-v)(X-\omega v) = \big(r(X) + \gamma_2(f^*(X) - \hat{f})\big)(X-\omega v) \\ + \gamma_3(z(X) - \hat{z})(X - v). \end{aligned} \tag{6}$$

Evaluating (6) at $X = v$ gives $(r(v)+\gamma_2(f^*(v)-\hat{f}))(v-\omega v) = 0$. Thus, if $f^*(v) \neq \hat{f}$, the above is a non-trivial linear equation in γ_2 with at most one solution, which occurs with probability at most $1/|\mathbb{F}|$. Hence, except with probability $1/|\mathbb{F}| + O(1/|\mathbb{F}|)$, acceptance in Step 11 implies $f^*(v) = \hat{f}$, which in turn gives $r(v) = 0$. Similarly, evaluating (6) at $X = \omega v$ yields $z(\omega v) = \hat{z}$. Consequently, except with probability at most $2/|\mathbb{F}| + O(1/|\mathbb{F}|) + \mathsf{negl}(\lambda)$, acceptance of Step 11 implies $r(v) = 0, f^*(v) = \hat{f}, z(\omega v) = \hat{z}$. Equivalently, defining

$$p(X) := (f^*(X) - \alpha)\, z(X) - (I(X) - \alpha)\, z(\omega X) + \gamma_1(z(X) - 1)\, \mathcal{L}_1(X) - Z_H(X)\, q(X)$$

we have $p(v) = 0$. Since v is uniform in $\mathbb{F} \setminus H$ after f^*, z, q are fixed, and $\deg(p) \leq 2N + 3$, the Schwartz–Zippel (Lemma 1) implies that $p(X)$ is the zero polynomial except with probability $(2N + 3)/(|\mathbb{F}| - N)$.

With $p(X) = 0$, evaluating at $X = \omega$ (where $\mathcal{L}_1(\omega) = 1$, $Z_H(\omega) = 0$) gives

$$(f^*(\omega) - \alpha)\, z(\omega) - (I(\omega) - \alpha)\, z(\omega^2) + \gamma_1(z(\omega) - 1) = 0.$$

If $z(\omega) \neq 1$, this is a non-trivial linear equation in the uniform challenge γ_1. Therefore, except with probability at most $1/|\mathbb{F}|$, we must have $z(\omega) = 1$.

Now evaluate $p(X) \equiv 0$ at each $X = \omega^i \in H$. For $i = 1$, since $Z_H(\omega) = 0$, $\mathcal{L}_1(\omega) = 1$, and we already proved $z(\omega) = 1$, we get $(f^*(\omega) - \alpha)z(\omega) = (I(\omega) - \alpha)z(\omega^2)$. For every $i \neq 1$, since $Z_H(\omega^i) = 0$ and $\mathcal{L}_1(\omega^i) = 0$, we get $(f^*(\omega^i) - \alpha)z(\omega^i) = (I(\omega^i) - \alpha)z(\omega^{i+1})$. Hence, for all $i \in [N]$, $(f^*(\omega^i) - \alpha)z(\omega^i) = (I(\omega^i) - \alpha)z(\omega^{i+1})$. Multiplying over all $i \in [N]$ gives

$$\textstyle\prod_{i=1}^N (f^*(\omega^i) - \alpha) \prod_{i=1}^N z(\omega^i) = \prod_{i=1}^N (I(\omega^i) - \alpha) \prod_{i=1}^N z(\omega^{i+1}).$$

Since $\omega^{N+1} = \omega$, the two z-products are the same, and since $I(\omega^i) = x^i$ for all $i \in [N]$, therefore $\prod_{i=1}^N (f^*(\omega^i) - \alpha) = \prod_{i=1}^N (x^i - \alpha)$. Let $a_i^* := f^*(\omega^i)$. If $\{a_i^*\} \neq \{x^i\}$ as multisets, then the two monic degree-N polynomials $\prod_{i=1}^N (T - a_i^*)$ and $\prod_{i=1}^N (T - x^i)$ are distinct. Since they agree at the random point $T = \alpha$, Schwartz–Zippel (Lemma 1) implies that this happens with probability at most $N/|\mathbb{F}|$. Therefore, except with probability $O(N/|\mathbb{F}|) + \mathsf{negl}(\lambda)$, the extractor recovers a permutation π^* such that $a_i^* = x^{\pi^*(i)}$ for all $i \in [N]$.

In Step 12, $\mathcal{A}$ executes Π_{Shuffle} on the same commitment C_f. By the witness-extended emulation of Π_{Shuffle} (Theorem 5), $\mathcal{E}$ extracts $(\boldsymbol{a}', \boldsymbol{\rho}^*, r_1', r_2')$ satisfying $\mathcal{R}_{\text{Shuffle}}$. By Theorem 2 (knowledge soundness in the AGM), the commitment C_f uniquely determines the polynomial $f^*(X)$, so $a_i' = f^*(\omega^i) = a_i^*$ for all i.

Combining the first two relations of $\mathcal{R}_{\text{Shuffle}}$ with $a_i^* = x^{\pi^*(i)}$ yields

$$\prod_{i=1}^N \mathsf{C}_i^{x^i} = \prod_{i=1}^N \tilde{\mathsf{C}}_i^{x^{\pi^*(i)}} \cdot \mathsf{Enc}_{pk}(1;\, -\rho_i^*)^{x^{\pi^*(i)}}.$$

Since x is sampled before $\mathcal{A}$ sends any message, Schwartz–Zippel (Lemma 1) implies $\tilde{\mathsf{C}}_i = \mathsf{C}_{\pi^*(i)} \cdot \mathsf{Enc}_{pk}(1;\, \rho_i^*)$ for all i, except with probability $\leq N/|\mathbb{F}|$. Taking a union bound over all failure events, the extractor fails only with negligible probability.

A.2 Proof of Theorem 5

Proof. We prove each property in turn.

Perfect completeness. Suppose $(\boldsymbol{a}, \boldsymbol{\rho}, r_1, r_2)$ is a valid witness for $\mathcal{R}_{\text{Shuffle}}$ and the prover is honest. The aggregated relation holds:

$$\begin{aligned} V_{\text{agg}} &= V_\ell \cdot V_r^w \cdot C_f^{w^2} \\ &= \left(\boldsymbol{h}_1^{\boldsymbol{a}} \cdot g^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle}\right) \cdot \left(\boldsymbol{h}_2^{\boldsymbol{a}} \cdot Y^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle}\right)^w \cdot \left(\boldsymbol{g}^{\boldsymbol{a}} \cdot \mathsf{u}_1^{r_1} \mathsf{u}_2^{r_2}\right)^{w^2} \\ &= (\boldsymbol{h}_1 \circ \boldsymbol{h}_2^w \circ \boldsymbol{g}^{w^2})^{\boldsymbol{a}} \cdot (g \cdot Y^w)^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle} \cdot (\mathsf{u}_1^{r_1} \mathsf{u}_2^{r_2})^{w^2} \\ &= \boldsymbol{h}_{\text{agg}}^{\boldsymbol{a}} \cdot U_{\text{agg}}^{\langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle} \cdot (\mathsf{u}_1^{r_1} \mathsf{u}_2^{r_2})^{w^2}. \end{aligned}$$

We verify that P as computed by the verifier in Step 7 equals $\boldsymbol{h}_{\text{agg}}^{\hat{\boldsymbol{l}}} \cdot \boldsymbol{k}^{\hat{\boldsymbol{r}}} \cdot U_{\text{agg}}^{\langle \hat{\boldsymbol{l}}, \hat{\boldsymbol{r}} \rangle}$ by expanding each factor and collecting contributions by generator:

- $\underline{\boldsymbol{h}_{\text{agg}}}$: V_{agg} contributes $\boldsymbol{h}_{\text{agg}}^{\boldsymbol{a}}$; A^u contributes $\boldsymbol{h}_{\text{agg}}^{\boldsymbol{s}_a u}$. Together: $\boldsymbol{h}_{\text{agg}}^{\boldsymbol{a} + \boldsymbol{s}_a u} = \boldsymbol{h}_{\text{agg}}^{\hat{\boldsymbol{l}}}$.
- $\underline{\boldsymbol{k}}$: D^{-1} contributes $\boldsymbol{k}^{-\boldsymbol{\rho}}$; A^u contributes $\boldsymbol{k}^{\boldsymbol{s}_\rho u}$. Together: $\boldsymbol{k}^{-\boldsymbol{\rho} + \boldsymbol{s}_\rho u} = \boldsymbol{k}^{\hat{\boldsymbol{r}}}$.
- $\underline{U_{\text{agg}}}$: V_{agg} contributes exponent $t_0 = \langle \boldsymbol{a}, -\boldsymbol{\rho} \rangle$; T_1^u contributes $t_1 u$; $T_2^{u^2}$ contributes $t_2 u^2$. Together: $t_0 + t_1 u + t_2 u^2 = t(u) = \langle \hat{\boldsymbol{l}}, \hat{\boldsymbol{r}} \rangle$.
- $\underline{\mathsf{u}_1}$: V_{agg} (via $C_f^{w^2}$) contributes $r_1 w^2$; A^u contributes $\rho_{r_1} u$; the term $\mathsf{u}_1^{-\mu_1} = \mathsf{u}_1^{-\rho_{r_1} u - r_1 w^2}$ cancels both. Likewise for u_2.
- $\underline{g}$: D^{-1} contributes $-\zeta$; A^u contributes δu; T_1^u contributes $\beta_1 u$; $T_2^{u^2}$ contributes $\beta_2 u^2$; $g^{-\mu - \beta_x} = g^{-(\delta u - \zeta) - (\beta_1 u + \beta_2 u^2)}$ cancels all of the above.

Hence $P = \boldsymbol{h}_{\text{agg}}^{\hat{\boldsymbol{l}}} \cdot \boldsymbol{k}^{\hat{\boldsymbol{r}}} \cdot U_{\text{agg}}^{\langle \hat{\boldsymbol{l}}, \hat{\boldsymbol{r}} \rangle}$ is a valid instance of $\mathcal{R}_{\text{IPA}}$, and Π_{IPA} accepts by its perfect completeness (Theorem 3).

Perfect special honest-verifier zero-knowledge. We construct a simulator $\mathcal{S}$ that, given the public statement, all verifier challenges $(w, u, \gamma_1, \ldots, \gamma_m)$, produces a transcript identically distributed to a real one.

1. Compute $\boldsymbol{h}_{\text{agg}}, U_{\text{agg}}, V_{\text{agg}}$ from the statement and challenge w.
2. Sample $\hat{\boldsymbol{l}}, \hat{\boldsymbol{r}} \xleftarrow{\$} \mathbb{F}^N$ and $\beta_x, \mu, \mu_1, \mu_2 \xleftarrow{\$} \mathbb{F}$ uniformly.
3. Sample $D, T_1, T_2 \xleftarrow{\$} \mathbb{G}_1$ uniformly.
4. Compute the IPA target $P^* := \boldsymbol{h}_{\text{agg}}^{\hat{\boldsymbol{l}}} \cdot \boldsymbol{k}^{\hat{\boldsymbol{r}}} \cdot U_{\text{agg}}^{\langle \hat{\boldsymbol{l}}, \hat{\boldsymbol{r}} \rangle}$, and solve for A:
$$A := \left(P^* \cdot V_{\text{agg}}^{-1} \cdot D \cdot T_1^{-u} \cdot T_2^{-u^2} \cdot \mathsf{u}_1^{\mu_1} \cdot \mathsf{u}_2^{\mu_2} \cdot g^{\mu+\beta_x}\right)^{u^{-1}}.$$
(u^{-1} exists since $u \neq 0$.)
5. Run Π_{IPA} honestly using witness $(\hat{\boldsymbol{l}}, \hat{\boldsymbol{r}})$ and challenges $(\gamma_1, \ldots, \gamma_m)$, producing the transcript $(L_1, R_1, \ldots, L_m, R_m, \hat{l}, \hat{r})$.

Perfect indistinguishability. Since u is sampled from $\mathbb{F} \setminus \{0\}$, the masked vectors $\hat{\boldsymbol{l}} = \boldsymbol{a} + \boldsymbol{s}_a u$ and $\hat{\boldsymbol{r}} = -\boldsymbol{\rho} + \boldsymbol{s}_\rho u$ are perfectly blinded by the uniform randomness of $\boldsymbol{s}_a$ and $\boldsymbol{s}_\rho$, and therefore uniformly distributed in $\mathbb{F}^N$. The scalars $\beta_x, \mu, \mu_1, \mu_2$ are likewise perfectly blinded by fresh uniform randomness. The commitments D, T_1, T_2 are perfectly hiding Pedersen commitments (Theorem 1), hence uniform in $\mathbb{G}_1$. The commitment A is uniquely determined by the verification equation; the simulator computes it identically. The IPA transcript is a deterministic function of $(\hat{\boldsymbol{l}}, \hat{\boldsymbol{r}})$ and the challenges; since these have identical distributions in the real and simulated executions, so does the transcript. The protocol is therefore perfect SHVZK.

Witness-extended emulation. Let $\mathcal{P}^*$ be a deterministic polynomial-time prover that makes the verifier accept with non-negligible probability ε. We construct an expected polynomial-time emulator $\mathcal{E}$ that extracts a witness $(\boldsymbol{a}^*, \boldsymbol{\rho}^*, r_1^*, r_2^*)$ such that $(\mathsf{ck}, V_\ell, V_r, C_f, \boldsymbol{h}_1, \boldsymbol{h}_2; \boldsymbol{a}^*, \boldsymbol{\rho}^*, r_1^*, r_2^*) \in \mathcal{R}_{\text{Shuffle}}$, except with negligible probability.

Step 1: fixed-w extraction. The emulator rewinds $\mathcal{P}^*$ to just before the verifier samples the aggregation challenge w, and obtains three accepting transcripts for three distinct challenges $w^{(1)}, w^{(2)}, w^{(3)} \in \mathbb{F}$, all sharing the same initial commitment D. For each fixed $j \in \{1, 2, 3\}$, let $(D, w^{(j)}, A^{(j)}, T_1^{(j)}, T_2^{(j)})$ be the corresponding first-round transcript. The emulator then rewinds inside this branch to obtain sufficiently many accepting continuations for distinct u-challenges, all sharing the same $(D, w^{(j)}, A^{(j)}, T_1^{(j)}, T_2^{(j)})$. For each accepting continuation with challenge $u^{(i)}$, the verifier computes the corresponding IPA target $P^{(i)}$. By the witness-extended emulation of Π_{IPA} (Theorem 3), the emulator extracts vectors $\hat{\boldsymbol{l}}^{(i)}, \hat{\boldsymbol{r}}^{(i)} \in \mathbb{F}^N$ such that $P^{(i)} = \boldsymbol{h}_{\text{agg}}^{\hat{\boldsymbol{l}}^{(i)}} \cdot \boldsymbol{k}^{\hat{\boldsymbol{r}}^{(i)}} \cdot U_{\text{agg}}^{\langle \hat{\boldsymbol{l}}^{(i)}, \hat{\boldsymbol{r}}^{(i)} \rangle}$. Since all accepting continuations in this branch share the same first-round messages $(A^{(j)}, T_1^{(j)}, T_2^{(j)})$,

and these Pedersen-type commitments are binding, the extracted IPA witnesses are consistent with a unique opening of the fixed first-round transcript. Hence there exist unique values $\boldsymbol{a}_j^* \in \mathbb{F}^N, \boldsymbol{\rho}_j^* \in \mathbb{F}^N, r_{1,j}^*, r_{2,j}^* \in \mathbb{F}$ such that

$$\begin{aligned} &V_\ell \cdot V_r^{w^{(j)}} \cdot C_f^{(w^{(j)})^2} \cdot D^{-1} = \\ &\boldsymbol{h}_{\mathrm{agg}}(w^{(j)})^{\boldsymbol{a}_j^*} \cdot \boldsymbol{k}^{-\boldsymbol{\rho}_j^*} \cdot U_{\mathrm{agg}}(w^{(j)})^{\langle \boldsymbol{a}_j^*, -\boldsymbol{\rho}_j^* \rangle} \cdot (\mathsf{u}_1^{r_{1,j}^*} \mathsf{u}_2^{r_{2,j}^*})^{(w^{(j)})^2}. \end{aligned} \tag{7}$$

Thus, for each $j \in \{1,2,3\}$, the emulator extracts values $\boldsymbol{a}_j^* \in \mathbb{F}^N$, $\boldsymbol{\rho}_j^* \in \mathbb{F}^N$, and $r_{1,j}^*, r_{2,j}^* \in \mathbb{F}$ satisfying (7).

Step 2: recover the original witness from the quadratic family in w. Define

$$F(w) := V_\ell \cdot V_r^w \cdot C_f^{w^2} \cdot D^{-1}.$$

For each $j \in \{1,2,3\}$, equation (7) gives

$$F(w^{(j)}) = \boldsymbol{h}_{\mathrm{agg}}(w^{(j)})^{\boldsymbol{a}_j^*} \cdot \boldsymbol{k}^{-\boldsymbol{\rho}_j^*} \cdot U_{\mathrm{agg}}(w^{(j)})^{\langle \boldsymbol{a}_j^*, -\boldsymbol{\rho}_j^* \rangle} \cdot (\mathsf{u}_1^{r_{1,j}^*} \mathsf{u}_2^{r_{2,j}^*})^{(w^{(j)})^2}.$$

Expanding $\boldsymbol{h}_{\mathrm{agg}}(w) = \boldsymbol{h}_1 \circ \boldsymbol{h}_2^w \circ \boldsymbol{g}^{w^2}, U_{\mathrm{agg}}(w) = g \cdot Y^w$, we obtain, for each j,

$$\begin{aligned} F(w^{(j)}) = {} & \boldsymbol{h}_1^{\boldsymbol{a}_j^*} \cdot \boldsymbol{h}_2^{w^{(j)} \boldsymbol{a}_j^*} \cdot \boldsymbol{g}^{(w^{(j)})^2 \boldsymbol{a}_j^*} \cdot \boldsymbol{k}^{-\boldsymbol{\rho}^*} \\ & \cdot g^{\langle \boldsymbol{a}_j^*, -\boldsymbol{\rho}^* \rangle} \cdot Y^{w^{(j)} \langle \boldsymbol{a}_j^*, -\boldsymbol{\rho}^* \rangle} \cdot \mathsf{u}_1^{(w^{(j)})^2 r_{1,j}^*} \cdot \mathsf{u}_2^{(w^{(j)})^2 r_{2,j}^*}. \end{aligned} \tag{8}$$

Now $F(w)$ is a quadratic family in w, and the bases $(\boldsymbol{h}_1, \boldsymbol{h}_2, \boldsymbol{g}, \boldsymbol{k}, g, Y, \mathsf{u}_1, \mathsf{u}_2)$ are independent. Hence equality of group elements is equivalent to equality of the corresponding exponent polynomials in w.

For each coordinate $i \in [N]$, the exponents of $\boldsymbol{h}_{1,i}$ and $\boldsymbol{k}_i$ in (8) are $(\boldsymbol{a}_j^*)_i$ and $-(\boldsymbol{\rho}_j^*)_i$, respectively. Since both $\boldsymbol{h}_{1,i}$ and $\boldsymbol{k}_i$ occur only in the constant term of $F(w)$, these exponents are independent of j. Hence there exist vectors $\boldsymbol{a}^*, \boldsymbol{\rho}^* \in \mathbb{F}^N$ such that $\boldsymbol{a}_1^* = \boldsymbol{a}_2^* = \boldsymbol{a}_3^* =: \boldsymbol{a}^*, \boldsymbol{\rho}_1^* = \boldsymbol{\rho}_2^* = \boldsymbol{\rho}_3^* =: \boldsymbol{\rho}^*$. Similarly, the exponents of u_1 and u_2 in (8) are $(w^{(j)})^2 r_{1,j}^*$ and $(w^{(j)})^2 r_{2,j}^*$. Since u_1 and u_2 occur only in the quadratic term of $F(w)$, the corresponding quadratic coefficients are independent of j. Hence there exist scalars $r_1^*, r_2^* \in \mathbb{F}$ such that

$$r_{1,1}^* = r_{1,2}^* = r_{1,3}^* =: r_1^*, \qquad r_{2,1}^* = r_{2,2}^* = r_{2,3}^* =: r_2^*.$$

Substituting these common values into (8), and then matching the constant, linear, and quadratic coefficients of $F(w) = V_\ell \cdot V_r^w \cdot C_f^{w^2} \cdot D^{-1}$, we obtain

$$V_\ell \cdot D^{-1} = \boldsymbol{h}_1^{\boldsymbol{a}^*} \cdot \boldsymbol{k}^{-\boldsymbol{\rho}^*} \cdot g^{\langle \boldsymbol{a}^*, -\boldsymbol{\rho}^* \rangle}, \quad V_r = \boldsymbol{h}_2^{\boldsymbol{a}^*} \cdot Y^{\langle \boldsymbol{a}^*, -\boldsymbol{\rho}^* \rangle}, \quad C_f = \boldsymbol{g}^{\boldsymbol{a}^*} \cdot \mathsf{u}_1^{r_1^*} \mathsf{u}_2^{r_2^*}.$$

Moreover, since D is fixed across the three outer rewindings and the commitment $D = \boldsymbol{k}^{\boldsymbol{\rho}} \cdot g^\zeta$ is binding, the value $\boldsymbol{\rho}^*$ identified above must coincide with the unique $\boldsymbol{\rho}$ component in the opening of D. Hence the factor $\boldsymbol{k}^{-\boldsymbol{\rho}^*}$ cancels against D^{-1} in the first identity, yielding $V_\ell = \boldsymbol{h}_1^{\boldsymbol{a}^*} \cdot g^{\langle \boldsymbol{a}^*, -\boldsymbol{\rho}^* \rangle}$. Thus all three defining relations of $\mathcal{R}_{\mathrm{Shuffle}}$ hold:

$$V_\ell = \boldsymbol{h}_1^{\boldsymbol{a}^*} \cdot g^{\langle \boldsymbol{a}^*, -\boldsymbol{\rho}^* \rangle}, \qquad V_r = \boldsymbol{h}_2^{\boldsymbol{a}^*} \cdot Y^{\langle \boldsymbol{a}^*, -\boldsymbol{\rho}^* \rangle}, \qquad C_f = \boldsymbol{g}^{\boldsymbol{a}^*} \cdot \mathsf{u}_1^{r_1^*} \mathsf{u}_2^{r_2^*}.$$

Therefore $(\mathsf{ck}, V_\ell, V_r, C_f, \boldsymbol{h}_1, \boldsymbol{h}_2;\ \boldsymbol{a}^*, \boldsymbol{\rho}^*, r_1^*, r_2^*) \in \mathcal{R}_{\text{Shuffle}}$. The emulator runs in expected polynomial time by standard rewinding, since it performs only polynomially many rewindings of a prover that accepts with non-negligible probability. Hence witness-extended emulation holds.